

Authentica: AI Powered Deep Fake Detection and Prevention

A major project report submitted in partial fulfilment of the requirement for
the award of degree of

Bachelor of Technology
in
Computer Science & Engineering

Submitted by

**Ritesh Dhiman (211138), Shwetabh Vikram (211178), Aavishkar
(211216)**

Under the guidance & supervision of

Dr. Deepak Gupta



**Department of Computer Science & Engineering and
Information Technology**
**Jaypee University of Information Technology, Wagnaghat,
Solan - 173234 (India),
May 2025**

SUPERVISOR'S CERTIFICATE

This is to certify that the major project report entitled '**Authentica: AI Powered Deepfake Detection and Prevention**', submitted in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology in Computer Science & Engineering**, in the Department of Computer Science & Engineering and Information Technology, Jaypee University of Information Technology, Waknaghat, is a bona fide project work carried out under my supervision during the period from July 2024 to May 2025.

I have personally supervised the research work and confirm that it meets the standards required for submission. The project work has been conducted in accordance with ethical guidelines, and the matter embodied in the report has not been submitted elsewhere for the award of any other degree or diploma.

(Supervisor Signature)

Supervisor Name: Dr. Deepak Gupta

Designation: Assistant Professor (SG)

Department: Dept. of CSE & IT

Date: 5 May 2025

Place: JUIT Solan

CANDIDATE'S DECLARATION

We hereby declare that the work presented in this report entitled '**Authentica: AI Powered Deepfake Detection and Prevention**' submitted in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology in Computer Science & Engineering** in the Department of Computer Science & Engineering and Information Technology, Jaypee University of Information Technology, Waknaghat is an authentic record of our own work carried out over a period from July 2024 to May 2025 under the supervision of **Dr. Deepak Gupta**.

The matter embodied in the report has not been submitted for the award of any other degree or diploma.

Name: Ritesh Dhiman

Roll No.: 211138

Date: 5 May 2025

Name: Shwetabh Vikram

Roll No.: 211178

Date: 5 May 2025

Name: Aavishkar

Roll No.: 211216

Date: 5 May 2025

This is to certify that the above statement made by the candidates is true to the best of my knowledge.

Supervisor Name: Dr. Deepak Gupta

Designation: Assistant Professor (SG)

Department: CSE & IT

Date: 5 May 2025

ACKNOWLEDGEMENT

Firstly, we express our heartiest thanks and gratefulness to almighty God for His divine blessing makes it possible for us to complete the project work successfully.

We are really grateful and wish our profound indebtedness to Supervisor **Dr. Deepak Gupta, Assistant Professor (SG)**, Department of Computer Science Engineering Jaypee University of Information Technology, Waknaghat.

We would also generously welcome each one of those individuals who have helped me straightforwardly or in a roundabout way in making this project a win. In this unique situation, we might want to thank the various staff individuals, both educating and non-instructing, which have developed their convenient help and facilitated our undertaking.

Finally, we must acknowledge with due respect the constant support and patience of our parents.

Ritesh Dhiman

(211138)

Shwetabh Vikram

(211178)

Aavishkar

(211216)

TABLE OF CONTENTS

LIST OF FIGURES.....	VI
LIST OF ABBREVIATIONS	VII
LIST OF TABLES.....	IX
ABSTRACT	X
1 INTRODUCTION	1
1.1 Introduction	1
1.2 Problem Statement.....	8
1.3 Objectives.....	8
1.4 Significance and motivation of the project report	9
1.4.1 Significance	9
1.4.2 Motivation	10
1.5 Organization of project report	11
2 LITERATURE SURVEY	12
2.1 Overview of relevant literature.....	13
2.1.1 A summary of the relevant papers.....	16
2.2 Key gaps in the literature.....	22
3 SYSTEM DEVELOPMENT	24
3.1 Requirements and Analysis	24
3.1.1 Functional Requirements.....	25
3.1.2 Non-Functional Requirements.....	26
3.1.3 Languages Used... ..	27
3.2 Project Design and Architecture.....	28
3.3 Data Preparation... ..	32

3.4	Implementation.....	33
3.4.1	Introduction... ..	33
3.4.2	Tools And Techniques.....	34
3.4.3	Algorithm... ..	37
3.5	Key Challenges.....	39
4	TESTING	48
4.1	Testing strategy.....	48
4.2	Test cases and outcomes.....	48
5	RESULTS AND EVALUATION.....	51
5.1	Results.....	51
5.2	Existing Deepfake Detection System... ..	62
6	CONCLUSIONS AND FUTURE SCOPE	64
6.1	Conclusion.....	65
6.2	Future scope.....	65
	REFERENCES.....	68 - 70
	APPENDIX.....	71

LIST OF FIGURES

Fig. No.	Title	Page No.
Fig. 3.1	Flow Chart of the Project	18
Fig. 3.2	Training Workflow	21
Fig. 3.3	Testing Workflow	22
Fig. 3.4	Sequence Diagram	24
Fig. 3.5	System Architecture	24
Fig. 3.6	Deep Fake Generation	25
Fig. 3.7	Face Swapped deepfake generation	26
Fig. 3.8	Dataset	34
Fig. 3.9	Frame Extraction and Preprocessing from Video	34
Fig. 3.10	Softmax Layer	36
Fig. 3.11	Deepfake Detection Model using ResNeXt50 and LSTM	36
Fig. 3.12	Overview of LSTM Architecture	37
Fig. 3.13	Deepfake Detection Model Using LSTM Layers	37
Fig. 3.14	Video Preprocessing	38
Fig. 3.15	Deepfake Detection Model Using LSTM Layers	43
Fig. 3.16	Training results with accuracy	44
Fig. 3.17	Training of the model	45
Fig. 3.18	Embedding visible watermark (FAKE)	46
Fig. 3.19	Embedding Invisible Watermark using LSB	46
Fig. 3.20	Deepfake Prevention	47
Fig. 4.1	Prediction function and file upload handling	49
Fig. 4.2	Model initialization and execution	50
Fig. 5.1	Training and validation accuracy over epochs	53
Fig 5.2	Training and validation loss over epochs	53

Fig 5.3	Confusion matrix	54
Fig 5.4	Uploading files	55
Fig. 5.5	Choosing the video	55
Fig. 5.6	Video in process of testing	56
Fig. 5.7	Final result of fake video	56
Fig 5.8	Final result of real video	57
Fig 5.9	Final result of self-made video	58
Fig. 5.10	Sample watermarked image (BOTTOM LEFT)	61
Fig. 5.11	Invisible watermarked image	61
Fig. 5.12	Deepfake image Prevented	62

LIST OF ABBREVIATIONS

Abbreviations	Meaning
ML	Machine Learning
CNN	Convolutional Neural Network
RNN	Recurrent Neural Network
LSTM	Long Short Term Memory
AI	Artificial Intelligence
GPT	Generative Pre-trained Transformer
GAN	Generative Adversarial Network
UML	Unified Modeling Language
DFDC	Deepfake Detection Challenge
FF	FaceForensics++ (Dataset)
CVPR	Conference on Computer Vision and Pattern Recognition
WACV	Winter Conference on Applications of Computer Vision
OpenCV	Open Source Computer Vision Library

LIST OF TABLES

Table No.	Title	Page No.
Table 2.1	Literature Review Table	16
Table 2.2	Test Cases	48 - 49
Table 2.3	Training and validation performance over epochs	51 - 52

ABSTRACT

The growing power of computation has made the algorithms so powerful in deep learning that creating such indistinguishable human synthesized video has been made extremely simple and thus become very common. So-called deep fake videos, switching faces or modifying them very convincingly in any video so they are becoming indistinguishable from originals, also grow in access and popularity with ease of creation. There will always be scenarios where realistic face-swapped deepfakes are used for political disturbances, like creating a deepfake audio of Kamala Harris or the Biden robocall during the 2024 US presidential election, fake terrorism events, revenge porn, blackmail people are easily envisioned. Now, with all the fake news becoming a potential challenge, add pleasantly credibly deepfake and it makes a time very difficult for a public or so to differentiate fact from fiction.

According to a survey by the global security software company McAfee, 22% of Indian people have recently stumbled on political deepfake videos and later recognized them as fake. It also highlighted operability concerns regarding the fact that deepfake technology can be misused. Respondents have a wide range of fear regarding it; 55% are worried about cyberbullying, whereas 52% fear fake pornographic content creation. In this work, we describe a new deep learning-based method for distinguishing AI-generated fake videos from real videos. Our method automatically identifies replacement and reenactment deep fakes.

We are trying to fight fire with fire by using AI. Our system uses Res-Next Convolution neural network in extracting the frame-level features that further used to train Long Short-Term Memory (LSTM) mentioned in Fig. 3.12, based Recurrent Neural Network (RNN) to classify that the video is subject to any kind of manipulation or not, i.e., it classifies whether the video is deep fake or a real video.

CHAPTER 1

INTRODUCTION

1.1 INTRODUCTION

Science has developed so much today, that there is no aspect of any particular interaction with people that has not been affected by it today. Just like the other advances, so is this keeping convenience alongside innovation, but it also introduces serious challenges to human life. Some of the biggest challenges in this light include the new emerging area of deepfake technology. Deepfakes are mostly defined as materials generated through AI, and they happen to mimic real individuals in most cases. They have just brought new opportunities and dangers in the field of digital security and trust. Whether the spread of misinformation or perpetrated fraud or compromised personal identities, the very misuse of deepfake technology raises huge ethical, social, or legal issues.

Deepfake threatens to go beyond the line of pure digital impersonation to strike at the very foundations of trust in visual and audio content. The illegal ramifications of such tools have increased manifestly in open sources while making it more important to make solutions to combat the indulgence viable. The old-fashioned methods of detection for forged material usually do not keep pace with the sophistication of current AI-based forgery tools. This is a glaring gap in the field of digital authenticity.

With Authentica: AI-Powered Deepfake Detection, however, it aims to come up with a mechanism that outmatches the others in handling automatic deepfake detection. An innovative artificial intelligence approach is employed to detect and prevent the automated processing of deepfakes. It comprises several advanced technologies like deep learning, computer vision, real-time monitoring, etc., so as to get more accuracy on manipulated content detection. With the use of innovative AI techniques, it detects and eliminates or prevents the automated processing of deepfakes.

It includes state-of-the-art technology such as deep learning, computer vision, real-time monitoring etc., to achieve accurate manipulated content detection. It also aims at detecting

automatic deepfake as well as gaining user trust through better transparency and safety in the platforms.

In the past few years, demand for the digital content authenticity verification tools has become alarmingly high. Authentica gives one the secure and safest channel to use. This project helps the user to move forward in the world of problem solving, these problems could be misinformation and theft of the identity. Ultimately to create the safer and trustable system for the safety of personal information. This project is to counter the effects of deepfakes technology, hence to make sure that the virtual world is the safer place to keep the user data or information safe.

In order to improve the efficiency of the Authentica, it is needed to be integrated with the ML model which is working on the principle of the statistical frameworks and mathematical computation which will ensure the accurate detection of deepfake. In this system the Convolutional neural networks (CNNs) are used and the best alternative of RNN which are used to detect the anomalies.

1.1.1 Mathematical Framework for Deepfake Detection

1. Feature Extraction and Analysis

The equation will start with feature extraction (Fig. 3.9), in this the unfiltered input such as frame s of video are presented in form of matrix, this matrix can also be the audio file. Let $X \in$

$R^{m \times n}$ denote an input image matrix where m and n represent pixel dimensions.

The CNN applies filters $F \in R^{k \times k}$ (where k is the kernel size) to detect features:

$$Y[i, j] = \sum_{p=0}^{k-1} \sum_{q=0}^{k-1} X[i+p, j+q] \cdot F[p, q]$$

Here the feature map in result is Y . This method with capture the spatial information which are necessary for identifying alterations.

2. Temporal Consistency in Videos

In case of video files, the key identification factor of deepfake is the temporal inconsistencies. Considering RNNs and Long short term memory (LSTM) are used to capture dependency over time. Given the sequence of frames like $\{x\}_{t=1}^T$, the LSTM equations are:

$$f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f)$$

$$i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i)$$

$$o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o)$$

Here, f_t , o_t , c_t , i_t are forget, input, output gates, and cell states respectively, and h_t is the hidden state. This will help the model to identify the irregularities around the frames.

3. Audio-Visual Synchronization

To identify the mismatch in the display and audio, correlation is computed. If $A(t)$ represents audio signals and $V(t)$ represents video signals:

$$\rho(A, V) = \frac{Cov(A, V)}{\sqrt{Var(A) \cdot Var(V)}}$$

Deepfake alteration is the clear sign of low correlation.

1.1.2 Advanced Techniques

1. Fourier Transform: The Fourier transform of an image $I(x, y)$ is:

$$F(u, v) = \sum_{x=0}^{m-1} \sum_{y=0}^{n-1} I(x, y) e^{-2\pi i \frac{(ux+vy)}{mn}}$$

Deepfakes often exhibit anomalies in high-frequency domains.

2. **Autoencoders:** these are supposed to be trained on true content as they form inputs accordingly. Remade error E behaves as the metric:

$$E = \|X - \hat{X}\|^2$$

Altered content means high value of E.

3. **Adversarial Training:** The GAN based models processes the attack to train the system. Here the D(x) and G(z) represent the output of discriminator and generator respectively.

1.1.3 Real-Time Implementation

This is one of the key ideas to make the model scalable around the places in need. In order to develop the kind of system we need to integrate the system of cloud computing and the edge devices. These devices can be the cameras. Also, we can put the live footage sin the social media network or live broadcasting system, Authentica make sures:

- Scalability through distributed processing frameworks.
- High throughput, enabling live detection for platforms with massive traffic.
- User-centric transparency via explainable AI techniques.

These requirements are fulfilled with the parallel processing and strategies with optimal model deployment.

1. Parallelized Feature Extraction

The task of feature extraction in which we apply the convolutional neural network are very intense when worked computationally. To look after this the GPUs are used in a distributive manner, dividing the input information across multiple GPUs for parallel processing. In Pipeline Parallelism The computation is divided into stages, each handled by a specific GPU, reducing memory overhead.

In given equation X is the input and $f_i(X)$ is the feature extraction for the Ith iteration. So, this is described below:

$$\text{Out} = \bigcup_{i=1}^N f_i(X)$$

Here is the GPU distribution.

2. Dynamic Model Compression

There are lot of parameters in the deep learning models around millions. Which makes them computationally very intensive. There are some techniques given below which decreases the size of the model without compromising to accuracy:

2.1 Pruning: Eliminates the weight w which is not of that much importance, where w is less than the epsilon which is a threshold.

2.2 Quantization: It decreases the bit precision from 32 bit floating point to 8 bit integers, Ultimately so that the storage and computation will be reduces for the model.

2.3 Knowledge Distillation: Here the difference between the models output which is considered to be one trainer and another learner is decreased.

3. Cloud-Based Scalability

Our model can be integrated with cloud platforms which will handle large scale data in real time:

3.1 Serverless Architecture: By itself it enhances the resource in the sequence of demand.

3.2 Content Delivery Networks (CDNs): This promises the processing will be fast by reserving the static features.

4. Explainable AI (XAI) Integration

Authentica is able to include XAI methods which can provide explainable results:

4.1 Feature Importance Visualization: Points out the regions in the frames which are most important for the decision.

4.2 Saliency Maps: It shows the areas of the frames which are influencing the output, it answers to user the why for each label.

5. Real-Time Detection Pipeline

For streaming services, the real time pipelining is very important, for optimization:

5.1 Batch Processing: Frames are grouped in batches, processes them simultaneously.

5.2 Sliding Window Analysis: Understands the sequences of frames for the temporal information.

Let $\{x\}_{t=1}^T$ shows the video frames and size of window:

Sliding Analysis: $f(x_t, x_{t+1}, \dots, x_{t+w-1})$

For final prediction every window output makes the contribution.

6. Advanced Security Measures

In order to prevent the misuse, the safety measures can be taken:

6.1 Blockchain Verification: For the record to be unchangeable the results are stored in blockchain entry.

6.2 Encrypted Communication: To protect the sensitive data certain transmission protocols can be used.

1.1.4 Long-Term Vision for Authentica

1. Addition with Regulatory frameworks

Authentica correlates with the legal and ethical standards, this will ensure that all the privacy laws are followed which are GDPR and CCPA. To avoid the misuse the collaboration with policymakers are important.

2. Multi-Modal Detection

In the future of the project we aim to add Natural language processing which will help to detect the altered captions in video. This multi-modal approach strengthens the system's capability to address misinformation.

3. Open-Source Community Contributions

By releasing parts of the framework as open-source, Authentica seeks to foster collaboration, enabling researchers to contribute to the ongoing development of deepfake detection techniques.

1.2 PROBLEM STATEMENT

The development of this technology in artificial intelligence is extremely fast, at the same time, the development of digital media technologies generates deepfake, an AI-generated synthetic media product that can imitate a real person almost realistically. Once the playthings of the entertainment programmers, today deepfakes are one of the most pernicious threats to the integrity of our digital information environment, and they are being abused more than ever in the service of misinformation, identity theft, revenge porn and financial fraud.

Deep Fake Concern: Damage inflicted on the organization caused by online content confusion of individuals and data privacy breach; the traditional techniques fail to detect highly advanced deepfakes generated using such high-level techniques as GANs. These in exploitation do prey upon the users. Such use may include the propagation of false ideologies, the misuse of personal likenesses without consent or financial damages caused by manner of fraudulent transfer of funds.

In response to these threats, Authentica developed a complete deepfake detection and response framework using state-of-the-art AIs. The algorithms struggle to analyze and identify photos, videos, and audio that are manipulated helps the contact network as they use new analysis and detection technology to be developed in real-time, but they are of course guided by user-friendly. All in all, while developing such a system, Authentica's mission is to secure the digital contents, safeguard privacy and endorse trust with the validity of online media.

The present project aims to create an efficient and transparent system that saves the world from being exposed to the threat of deepfakes while securing a sense of integrity in digital media and preventing it from being mis utilized for ulterior purposes.

1.3 OBJECTIVES

With this project, AUTHENTICA: AI-powered Deepfake Detection and Prevention, we aim to tackle the growing concerns surrounding deepfake technologies by developing a comprehensive system for their detection and prevention. The project seeks to fulfill the following key objectives

1. The ultimate aim is to build a very intelligent AI that could very accurately catch the deepfakes at least in video, audio, and images. This will provide for state-of-the-art machine learning-based algorithms for detection of altered contents with an emphasis on extendibility and accuracy. Deepfake detection features can cover all dimensions that include exaggerated and subtle manipulations. For verifying the digital content, this could be a one-stop solution to making users feel free to trust or verify it.
2. The main aim is to have an exceptionally developed AI that could detect deepfakes in terms of video, audio, and images, using state-of-the-art machine learning techniques for detection of altered content with emphasis on scale and accuracy. This will be a one-stop solution of all factors of detection and then verification of digital media with effectively high sensitivity to manipulate exaggerated and rather subtle forms as well.
3. Multi-modal analysis considers audio, video, and textual data for more accurate detection. It picks up on modal discrepancies like mismatched lip movements,

unnatural audio, and inconsistent text. The use of different modalities ensures an integrated deep fake detection approach, particularly in cases where manipulations are performed over multiple modalities at once.

4. The proposed system will refine existing detection algorithms and employ ensemble models to mitigate the chances of producing false positive and negative results. These errors threaten the reliability of detection and can trigger false alarms and missed manipulations. Feature optimization and cross-validation techniques will be applied to reduce these errors and make the system reliable for application in real-time.
5. For the power performance of deepfake detection models (Fig. 3.11), various datasets are required. This project develops and expands the datasets with different authentic and manipulated face data such as variations in expressions, languages, illumination, and many others. An all-inclusive dataset would be ideal to train such models that generalize well on unseen contents.

1.4 SIGNIFICANCE AND MOTIVATION OF THE PROJECT

1.4.1 SIGNIFICANCE

The importance of our project lies in its ability to tackle the urgent and growing problem of deepfakes, which pose serious risks to digital security, media integrity, and public trust. Deepfake uses the advanced frameworks to change the video and audio file which make it look very accurate and realistic. Earlier used methods of identifying it are not able to keep up with the complexity of the AI which is used in today's deepfake technology.

Through this project we aim to solve the problem of deepfake. We will use the advanced machine learning techniques. It finds very minute details such as facial expression, voice matching with the video and the patterns found in the frames. By doing this it will help in detecting the deepfake more accurately and will reduce the chances of people getting misled by the altered videos and audio. It will reduce the fraud with the information and stop the damage led to someone's reputation.

The real utility of this project is coming in front when the preventive measures are introduced within the detection system, this will give real time mechanisms to have a protection against the deepfakes. In the field of journalism, entertainment, and private life it is very much needed and in urgency. Its scalability and applicability makes it a base for future experimentation and enhancements in regards to digital media verification or AI ethics.

1.4.2 MOTIVATION

Biggest motivation arises from the fact that the deepfake technologies are at their potential and verge to disrupt the social, political and economic systems, Deepfakes carries a potential of spreading misinformation, destroying someone's reputation, increasing cybercrime. So, detecting them is the top most priority.

In the world of media and journalism, the power to separate original and altered data is important for combating fake news, digital misinformation and saving the credibility of

information sources. Agencies of law need strong tools to identify these deepfakes. It will help them stop the identity theft, blackmailing, or fraud which will ensure the justice to society. In the entertainment industry, deepfakes are very common but which the fans of the celebrity are provoked and all.

At our own level, people are losing their safety of digital information and in the time where data is considered very precious than the gold. So, it's very important that personal information should be protected and make the digital world secure.

Furthermore, as deepfake technology advances, so too must the countermeasures. This project not only addresses the current landscape of deepfake threats but also establishes a foundation for future research and development in AI-driven media forensics. By contributing to a more secure and trustworthy digital environment, Authentica is set to make a significant impact across various domains, from public policy to technological innovation.

1.5 ORGANIZATION OF PROJECT REPORT

The detailed organization of a project can be outlined as follows:

Chapter 1: Introduction

This chapter of the project report covers the topics such as the problem statement and the main objectives of the project. It also contains the significance of this project along with the motivation for this topic. Certain basic terminologies to detect the deep fake models.

Chapter 2: Literature Survey

This chapter of the project report covers the study that we did when we started working on this project. It contains the information about the research papers that we read to expand our knowledge regarding the topic and the previous work done in the field of Deep Fake Detection and Prevention along with the current status, opportunities and challenges to these Detection models in future. It also covers the key challenges and limitations of each of the research papers which we went through.

Chapter 3: System Development

This particular chapter of the project report contains the various functional and non-functional requirements of the project. It also contains the design and architecture of this project. It also contains the implementation part of the project along with the key challenges that we faced during the project.

Chapter 4: Testing

This chapter of the project report covers the testing and validation part of the project. It contains the testing strategy that we used along with the test cases and their respective outcomes. This part of the report shows the various types of inputs provided by us to the chatbot and the respective outcomes of those inputs. This would show whether our chatbot works as expected by us or not.

Chapter 5: Results and Evaluation

This particular chapter of the report solely contains the results of the things implemented in the previous chapters. It contains the images of the outputs of the project along with the description.

Chapter 6: Conclusions and Future Scope

This is the final chapter of the report which talks about the end product of our amazing journey so far. It contains the conclusion of our report along with the scope of further improvements in the field of our project generally and chatbot specification.

CHAPTER 02:

LITERATURE SURVEY

2.1 OVERVIEW OF RELEVANT LITERATURE

B. Jacobsen [1] Presented explored algorithmic techniques for detecting deepfakes and evaluated their performance on various datasets. It highlighted that advanced detection algorithms demonstrate high accuracy in identifying manipulated content. However, the study noted potential biases in the training datasets, which could affect the reliability of the algorithms in real-world applications.

W. Yang [2] Presented an approach called "AVoiD-DF," which utilizes audio- visual joint learning to detect deepfakes. The method was tested using the DefakeAVMiT dataset, achieving effective detection of multi-modal deepfakes. Nevertheless, the study identified a limitation in the model's ability to generalize across diverse datasets, potentially restricting its application in broader contexts.

Z. Gu [3] Research introduced a spatiotemporal neural network designed to detect temporal inconsistencies in deepfake videos. The model was evaluated using a benchmark dataset and demonstrated significant effectiveness in detecting such inconsistencies. However, similar to other works, this approach faced challenges in generalizing its results to other datasets.

L. Trinh [4] Research focuses on creating a trustworthy deepfake detection using certain prototypes. In this the resnet is used as a fundamental model and it resulted in the enhanced interpreting in the detection of the deepfake. Apart from the improvement shown by the model the size of the dataset is the major issues in there, it will affect the scalability and the strength of the result.

A. Mehra [5] Research resulted in the identification of the applications of the capsule networks to increase the accuracy of the deepfake detection. Here it is noticed that the capsule network is able to handle and effectively capture the spatial dependencies within the

images, enhancing detection performance. However, the method faced challenges in computational complexity, which significantly increased training time.

X. Yang [6] Presented the method to identify the source of deepfake network and this is done by the method of the Source attribution network. However, it noted that source attribution might fail when dealing with highly modified or synthetic content, limiting its robustness in diverse scenarios.

Z. Hanqing [7] Presented a Multi-Attentional Mechanism that focused on key attentional regions in deepfake videos. Tested on the Deepfake Detection Dataset, the approach achieved state-of-the-art performance. However, its efficacy was limited in handling low-resolution videos, reducing its applicability in certain real-world cases.

R. Durall [8] Applied Variational Autoencoders combined with simple features and one-class classification for deepfake detection. It demonstrated strong performance in unsupervised learning, achieving high accuracy on small datasets. Nevertheless, the method struggled when applied to high-complexity datasets with large variance, which hindered its scalability.

S. Rana [9] Conducted a comprehensive review of multiple datasets and techniques used for deepfake detection. It identified trends and existing gaps in detection methods, highlighting the lack of robust real-time detection tools as a critical limitation.

K. Shiohara [10] Introduced the use of self-blended image generation to improve detection robustness. By blending real and fake images during training, the approach enhanced detection accuracy. However, it focused primarily on the face region, leaving the consistency of the entire image unaddressed.

H. Qi [11] Proposed the DeepRhythm technique, leveraging Remote Visual Photoplethysmography (rPPG) to monitor heartbeat rhythms. Using FaceForensics++ (Fig. 3.8) and DFDC-preview datasets, the method applied spatial-temporal attention, achieving superior results with models like Xception and MesoNet. However, its focus on facial heartbeat rhythms posed limitations in dealing with heavily compressed or blurred video data.

I. Amerini [12] Utilized optical flow fields combined with a CNN architecture to detect deepfake videos by analyzing temporal features. The method, tested on the FaceForensics++ dataset, achieved high accuracy but required significant computational resources, making it less accessible for widespread deployment.

D. Guéra [13] Explored the use of CNNs combined with RNNs for deepfake video detection, utilizing the HOHA dataset. The study achieved 97% accuracy with RNN-based models, highlighting their strength in temporal sequence analysis. However, the method's performance heavily relied on the quality of the input videos, limiting its robustness across diverse scenarios.

O. Ismael [14] Introduced a video tampering dataset, specifically designed for forensic investigations. It provided a comprehensive resource for analyzing tampered video content. However, the dataset lacked diversity in tampering techniques, reducing its applicability to more varied real-world forensic scenarios.

B. Dolhansky [15] Presented the DFDC dataset, a benchmark dataset comprising around 5,000 videos. By using models like XceptionNet and TamperNet, this study is able to make 93% accuracy in detection. Apart from being its higher utility dataset shows the limited diversity in deepfake detection method potentially becoming a obstruction to newer, more advanced methods.

K. Popat [16] Presented to make work of multiple datasets for evaluation and to evaluate the credibility of the claim the evidence aware deep learning model is taken into consideration. It is highly effective in integrating the external evidence in credibility evaluations. Due to its high computation, it faced many challenges and have some issues in large scale deployment.

B. Dolhansky [17] Presented the DFDC dataset which has approx. 5000 videos, which has been a mark for deepfake detection. This study has achieved 93% accuracy using the XceptionNet model and pointing the scope of model comparison. Apart from this the limit in creation of deepfake method restrict the applicability to improved techniques.

P. Korshunov [18] Introduced an efficient method for detecting fake images using frequency domain analysis and simple features. It demonstrated effective detection with minimal data requirements. However, the approach struggled to generalize well to lower- resolution images, which reduced its robustness in practical scenarios.

D. Guera [19] Employed CNNs and RNNs on the Deepfake Video Dataset, achieving a 94% accuracy rate. The RNN architecture focused on temporal features for effective deepfake detection. However, the method was limited to analyzing short (2-second) video sequences and was not tested on real-time streaming data.

R. Raghavendra [20] Utilized gaze tracking and face recognition to detect manipulations with high accuracy. It proved effective in differentiating between real and fake faces. Nevertheless, relying solely on gaze tracking was insufficient for detecting deepfake manipulations (Fig. 3.6 and Fig. 3.7) in more complex videos.

2.1.2 A SUMMARY OF THE RELEVANT PAPERS

Here in Table 2.1 we have provided a summary of some research papers that we have gone through over the past months, firstly in the form of a table and elaborated those in brief afterwards.

Table 2.1: Summary of the Relevant Literature

S. No	Author & Paper Title (Citation)	Journal/Conference (Year)	Tools/Techniques/ Dataset	Key Findings / Results	Limitations/Gaps Identified
1.	B. Jacobsen Deepfakes The Promise of Algorithmic Detectability [1]	International Journal of AI (2024)	Algorithmic detection techniques, various datasets	High accuracy in detecting deepfakes	Potential biases in algorithm training datasets
2.	W. Yang AVoid-DF: Audio-Visual Joint Learning for Detecting Deepfake [2]	IEEE Transactions on Information Forensics and Security (2023)	Audio-visual joint learning, DefakeAVMiT dataset	Effective detection of multi-modal deepfakes	Limited generalization across diverse datasets
3.	Z. Gu	ACM Multimedia	Spatiotemporal neural	Effective detection	Limited generalizability across

	Spatiotemporal Inconsistency Learning for DeepFake Video Detection [3]	(2021)	networks, benchmark dataset	of temporal inconsistencies	datasets
4.	L. Trinh Interpretable and Trustworthy Deepfake Detection via Dynamic Prototypes [4]	WACV (2021)	Dynamic prototype generation, ResNet	Improved interpretability of deepfake detection	Dataset size limitation
5	A. Mehra Use of Capsule Network to Detect Deepfake Videos [5]	IEEE Conference (2021)	Capsule Networks, FaceForensics dataset	Capsule networks improve detection accuracy	Computational complexity increases training time
6	X. Yang	IEEE (2021)	Source Attribution	Enables source	Source attribution may fail with

	Deepfake Source Detection [6]		Network, Deepfake Video Dataset	detection of deepfakes, achieving competitive results	highly modified or synthetic content.
7	Z. hanqing Multi-Attentional Deepfake Detection [7]	CVPR (2021)	Multi-Attentional Mechanism, Deepfake Detection Dataset	Achieves state-of-the-art performance by focusing on key	Does not perform well on low-resolution
8	R. Durall Classifying Deepfakes Using One-class Variational Autoencoder [8]	Variational Autoencoder, Simple Features, One-class Classification	Variational Autoencoder, Simple Features, One-class Classification	Detects deepfakes with unsupervised learning, achieving high accuracy on small datasets.	Struggles with high-complexity datasets with large variance.
9	S. Rana Systematic Review of	Journal of Information Security (2020)	Review of multiple datasets and	Identified trends and gaps in	Lack of robust real-time detection tools

	Deepfake Detection [9]		techniques	detection methods	
10	K. Shiohara Detecting Deepfakes with Self-Blended Images [10]	CVPR (2020)	Self-blended image generation,	Increases robustness by blending real and fake images for training, improves detection accuracy.	Focuses primarily on face region, not whole image consistency.
11	I. Amerini Deepfake Video Detection through Optical Flow Based CNN [11]	ICCV Workshop (2019)	Optical flow fields, CNN, FaceForensics++ dataset	Accurate detection of deepfake videos using temporal features	Requires high computational resources
12	D. Guéra Deepfake Video Detection Using Recurrent Neural	IEEE International Conference (2019)	CNN and RNN, HOHA dataset	97% accuracy in detecting deepfakes with RNN-based	Performance depends on video quality

	Networks [12]			model	
13	O. Ismael Video Tampering Dataset for Forensic Investigation [13]	Forensic Science International (2019)	Video tampering dataset, forensic investigation techniques	Provides a comprehensive video tampering dataset used in real-world forensic investigations.	Video dataset lacks diversity in tampering techniques.
14	B. Dolhansky The Deepfake Detection Challenge (DFDC) Preview Dataset [14]	ArXiv (2019)	DFDC Dataset, XceptionNet, TamperNet	Achieves 93% accuracy with XceptionNet.	Limited diversity in deepfake creation methods, may not generalize to newer techniques.
15	K. Popat DeCLarE: Debunking Fake News and False Claims	ACM Transactions (2018)	Evidence-aware deep learning model, multiple datasets	Effective credibility assessment with	High computational overhead

	using Evidence-Aware Deep Learning [15]			external evidence	
--	--	--	--	-------------------	--

2.2 KEY GAPS IN THE LITERATURE

Despite the great strides made on deepfake detection and prevention, gaps in the literature reveal that there are serious challenges to be overcome when developing strong and comprehensive solutions. Such gaps must therefore be filled to strengthen responses against the emerging threat of deepfakes.

2.2.1 Insufficient Robustness to Advanced Generative Models:

Deepfake technology advanced at a fast pace with StyleGAN3, DeepFaceLab, and advanced multimodal frameworks. The detection mechanisms used are often outdated due to the fast pace at which these models are progressing. Most studies are based on datasets or techniques focused on older generative adversarial networks (GANs), so their effectiveness is limited compared to the hyper-realistic outputs of newer systems. It is thus important to adapt to models that can handle this complexity in generative technologies.

2.2.2 Real-World-Applicability:

While promising results come from laboratory settings, it is still challenging to deploy deepfake detection algorithms in real-world applications. The environmental variables such as lighting, resolution, background noise, and compression artifacts may all degrade the performance of models considerably. Current models have very high false-positive or false-negative rates when deployed in real-world applications outside of the controlled environments. This gap, therefore, calls for methods that generalize well across diverse datasets.

2.2.3 Lack-of-Preventive-Measures-Integration:

Much of the literature focuses on post-creation detection, overlooking the potential of proactive approaches. Preventive measures, such as digital watermarking, blockchain for content verification, or cryptographic signatures, are underexplored. Combining these technologies with detection frameworks could prevent the creation or distribution of deepfakes altogether, but limited research has been directed toward such integration.

2.2.4 Limited Multimodal Detection Research:

Deepfake manipulation often targets multiple modalities, including video, audio, and text. Most existing studies specialize in detecting anomalies within a single modality, failing to develop comprehensive solutions for multimodal deepfakes. For example, a fake video with authentic audio could bypass detection systems focused solely on video anomalies. Research on unified frameworks capable of detecting manipulations across modalities remains inadequate.

2.2.5 Ethical and Legal Considerations:

The literature lacks sufficient attention to the ethical implications of deploying detection systems. Problems such as rights to privacy, misuse of the detection model, and the impact of society over false accusations. The legal tools to maintain the use of deepfake technologies and detection tools are still need to be improved. This is creating the gap between the technology and the legal policies.

2.2.6 Scalability and Real-Time Detection:

In social media, when it comes to the large volumes of content then its scalability is rare. Real time detection is very important to stop the spread of harmful deepfakes. However, there are some researches and studies which tells us about the computational speed which is in fact essential for the deepfake detection system. By looking into such problems, we will be able to develop a system which is effective, ethical and resilient.

CHAPTER 3

SYSTEM DEVELOPMENT

This chapter will portray a comprehensive discussion for the requirement which include both functional and nonfunctional needs. The architecture on which the deepfake detection and prevention works uses enhanced AI techniques, which includes CNNs and RNNs. This project will also have Prevention techniques in which watermarking system is used to prevent the video from being uploaded in any place.

In this chapter there is a comprehensive step by step system design and implementation process detail. For more understanding diagrams related to system architecture, data flow, UML, and use case diagrams are also mentioned. These figures will present the addition of different technologies at different stages of the project.

In this chapter one will get to know about the implementation phase. It emphasizes on the development of the powerful AI model that can easily identify the altered images, videos and audio files. The incorporated Long short-term memory (LSTM) model is being trained on datasets like face forensics and to enhance accuracy and reliability tools like TensorFlow[28] and Open Source Computer Vision Library (OpenCV)[29] is taken into use.

In this chapter it is also mentioned that what problems are faced during implementation of this project, which include difference in the dataset, computational requirements, scalability of the project and environment for practical deployment. These are discussed with strategies thus leading a way for development and enhancement. The discussion covers all methodological aspects of the project and addresses some of the technical and practical issues regarding deepfake detection and prevention.

3.1 REQUIREMENTS AND ANALYSIS

3.1.1 Functional Requirements

Functional Requirements

The Authentica: AI-powered Deepfake Detection and Prevention System is aimed at being the most powerful system meeting all the software and hardware requirement specifications. System specifications, architectural design and component-level analysis are the basis of the project, thereby ensuring hard-hitting deepfake detection and prevention measures. The functional requirements as detailed below:

- 1. Hardware:** The project requires hardware that is computationally efficient to run deep learning models for performing deepfake detection in images, videos, and audio. Complex models will require GPUs like NVIDIA RTX 3080,3090, Tesla A100 for handling the training as well as the inference. It is possible to perform parallel processing of big datasets and make the most of efficient implementations of AI algorithms only with these GPUs. In situations of very demanding tasks, the resources can be managed and the cloud scalability availed from GPU services using AWS, Azure, or Google Cloud.
- 2. Software:** Essentially, the core detection algorithms for deepfake detection are built on deep learning frameworks such as TensorFlow and PyTorch[30] since they lend flexibility to build custom architectures and integrate pre-trained models for further performance improvement OpenCV addresses preprocessing of media files, while librosa and NLTK do the work of audio and text analysis, respectively. Django is the prime backend framework for building robust APIs, whereas React.js will be used in making the user interface.
- 3. RAM:** The system requires a minimum of 16 GB RAM for local development and testing and preferably 32 GB and above for the handling of larger datasets while reducing memory bottlenecks. This guarantees smooth transition while training and evaluating deep learning models while maintaining efficient levels of processing during runtime.

4. **Storage:** Sufficient storage was required to manage datasets, model weights, and logs. SSDs were preferred for faster data access and reduced latency, ensuring efficient storage and retrieval during project execution.
5. **Network:** A high-speed and trust worthy internet connectivity was vital for the download of pre-trained models, datasets, and libraries as well as leveraging cloud-based computational resources. Since updates of some datasets were necessary in real-time or even access to online repositories, a sound network infrastructure was important for monitoring uninterrupted project development.
6. **Development Environment:** The development environment was based on Python programming with libraries such as TensorFlow, PyTorch, and OpenCV. It was specifically designed for deep learning applications, ensuring compatibility with the most-needed tools and APIs. The setup involved using an Integrated Development Environment like PyCharm or using Jupyter Notebooks to make code development easier, debug with nice visualization. This would provide an integrated framework both for model design and experimentation with it.
7. **Scalability:** Scalability was one of the utmost considerations, especially in terms of training (Fig. 3.1 and Fig. 3.2) and testing (Fig. 3.3) models on larger datasets or making the solution more adaptable for real-world applications.

3.1.2 NON-FUNCTIONAL REQUIREMENTS

1. **Accuracy:** The accuracy of the system is what matters, and this is what ensures the detection of deepfakes so that the results produced do not contain many false positives and false negatives. The detection algorithms must, therefore, be optimized in order to distinguish real content from manipulated media with high precision. High accuracy builds the confidence of the users and positions the system as a reliable tool against misinformation and fraud. Upon reaching this degree of accuracy, the system will fulfill its basic purpose and provide trustworthy outputs to its users.
2. **Performance:** In heavy workload it is important to ensure that the performance of the solution remain efficient and responsive. The time to detect the deepfake should be fast enough to take actions on time. The Latency should be less to let the user quickly retrieve

their results. In the it should be computational enough to support the multiple data files sent for examination. Optimization is essential to achieve these performance targets so that to have a seamless user experience.

3. **Scalability:** In this factor there should be ensures that that the large volumes of dataset is possible to be accommodated along with the platform needs. Simply it should be able to handle the large dataset with meeting all the needs of user without compromising the experience and accuracy. In this the architecture is not allowed to follow the horizontal scaling, which takes the additional computational power. Overall, the scalability ensures the system should remain effective even in heavy workload.
4. **Security:** The system is required to protect the information and data of the user and also prevent the data from getting misused. Encryption techniques must be used to protect the in transit and at rest data, simply to protect the sensitive information. Also, the system is required to follow the digital privacy guideline as mentioned by the Act under the laws made by government of India like GDPR. These steps will ensure that the user data will remain private and unbreachable.
5. **Flexibility:** Flexibility is considered as the adaptation in which the model is extended beyond in its computation. Alongside deepfake technology the model or an architecture should allow for the new algorithms and techniques to work in stopping the new threats emerging and considered to be contemporary to the deepfake technology ultimately it is required to remain completely aware of changing digital time sand taking action according to it.
6. **Maintainability:** This means that by the time passes the system developed should remain efficient and operational. In this architecture constant updates are required to be made such as bug fixing and improving the performance of the system. Coding practices and proper testing routine is to be incorporated to achieve the maintainability factor. Along with this modular design can also be improved. This allows the developers to implement the changes and look for the updates without compromising the main function of system.
7. **Usability:** It can be focused on providing the platform which is accessible and intuitive to all the user and from any domain. For this system should have clear instruction and easy

Navigation and results which are enough to ensure that the non-technical domain person can easily work on the system. It should have graphical representation for better understanding. Through this method the system will become more practical and have large range of useability.

- 8. Cross-platform Compatibility:** In this it is made sure that the system should work on the multiple platforms, which include web, smart phones. The system is needed to operational in various operating system and devices with a slight change in the system manually which is required. This will also increase the scalability of the project. Among multiple user it will have an experience of the diversity it carries. Overall, it will make project more appealing to the user.

3.1.3 SOFTWARE REQUIREMENTS

The requirements for the software are mentioned below:

3.1.3.1 LANGUAGES USED

- 1. Python:** Python is the fundamental in the journey of system development since it acted as the catalyst in implementation of the machine learning and deep learning model which are needed for the deepfake detection and prevention. There are certain frameworks which are used which act as an essential player in designing, training and optimization of the model. These are Tensorflow and PyTorch. Also, python has extensive collection of libraries which makes the handling and preprocessing very efficient. Overall python has a dominance over the computational processes which makes it more precise and scalable.

3.1.3.2 Libraries Used

There are three frameworks:

- 1. TensorFlow and PyTorch:** With the following mentioned tools we can create the deep learning models. This gives various benefits such as flexibility, scalability, giving help to accuracy and efficient of the system. Also, the pretrained model and the custom architectures will increase the development of the model without the need of high performance.

2. **OpenCV:** OpenCV is essentially important in processing images and video files. It can carry out operations such as frame capture and editing. It guarantees the efficiency of the whole pipeline for detection purposes and smooth integration with it.
3. **Numpy and Pandas:** NumPy and pandas are essential libraries for preprocessing and analysis of data. They enable easy manipulation of datasets, thus ensuring that the model training and evaluation process are handled efficiently in terms of numerical and structured data. The libraries simplify the workflows for preprocessing and optimize the system performance.

3.2 PROJECT DESIGN AND ARCHITECTURE

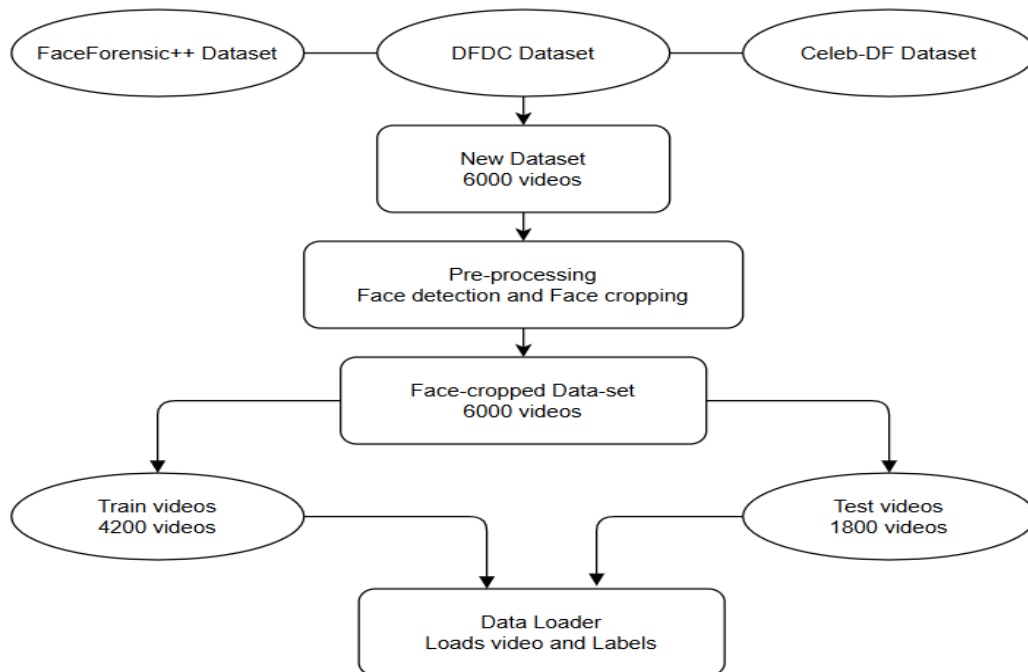


Fig. 3.1. Training workflow

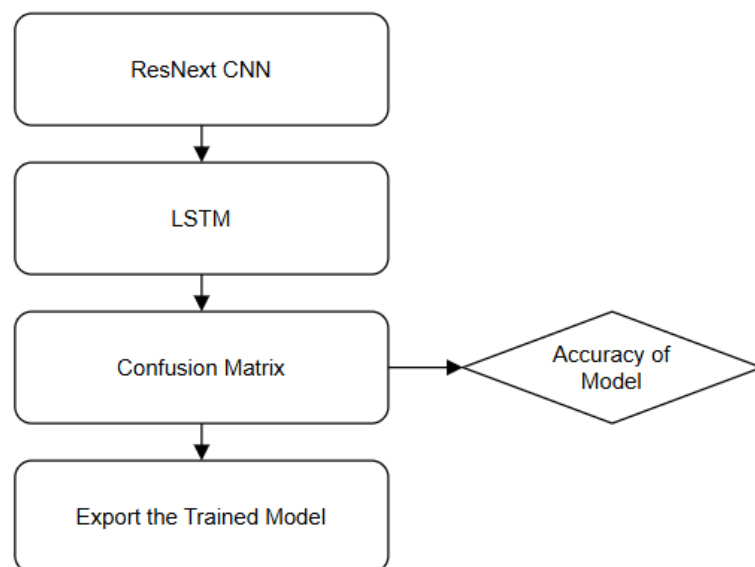


Fig. 3.2. Training Workflow

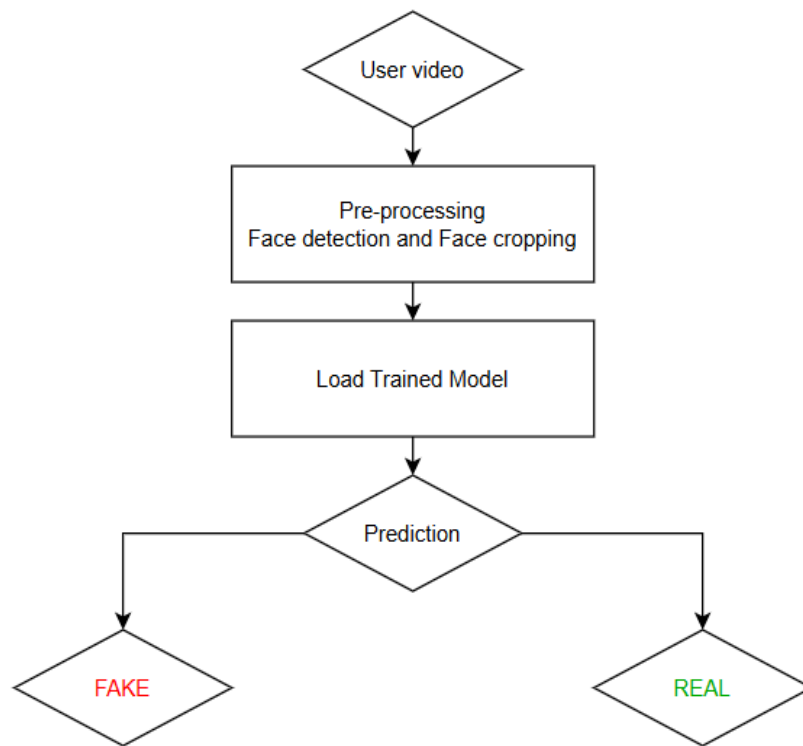


Fig. 3.3. Testing Workflow

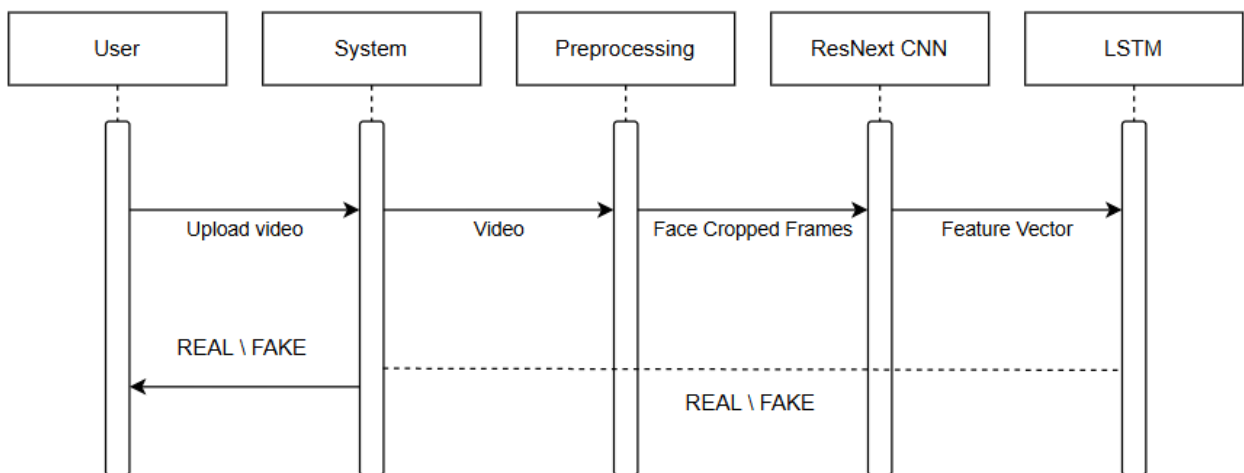


Fig. 3.4. Sequence Diagram

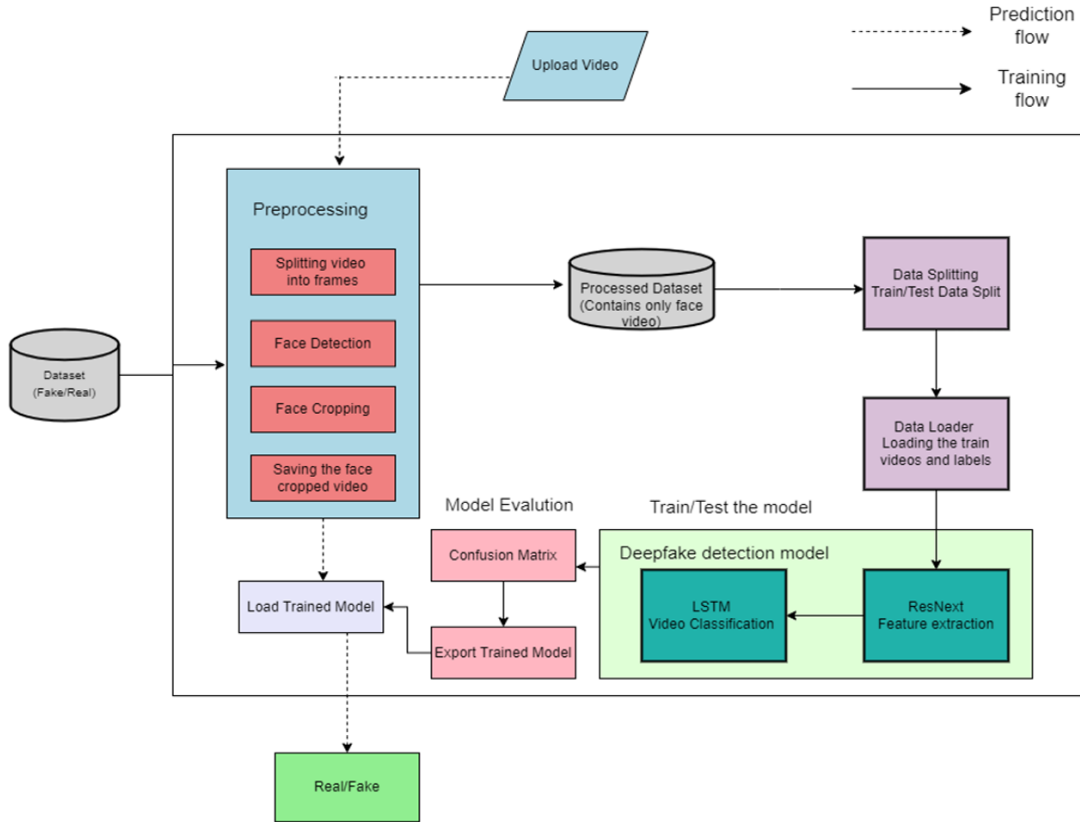


Fig. 3.5. System Architecture

This is a system that has avoided the bias of the model that will be used. It will train our PyTorch deepfake detection model with the number of real and fake videos on par to avoid model bias. The architecture of the system model is demonstrated in Fig. 3.5 and Sequence diagram is shown in Fig.3.4. In the development, we have Deepfake Video Detection taken the dataset that we have preprocessed with it and developed a new processed dataset, which included only face-cropped videos.

3.2.1 Creating deepfake videos

It is very important to understand the creation process of the deepfake in order to detect the deep fake videos. Majority of the tools, including the GAN and autoencoders, takes a source image and target video as input. These tools split the video into frames, detect the face in the video, and replace the source face with the target face on each frame.

Then the replaced frames are combined using different pre-trained models. These models also improve the quality of video by removing the left-over traces by the deepfake creation model. Which result in creating a deepfake looks real in nature. We have used the same approach to detect the deepfakes also. The deepfakes created using the pretrained neural networks models are so realistic that it is nearly impossible to spot the difference with the naked eyes. But in reality, the deepfakes creation tools leave some traces or artifacts in the video, which may not be noticed by the naked eyes. The motive of this paper identifies these unnoticeable traces and distinguishes these artifacts of these videos and classified it as deepfake or real video.

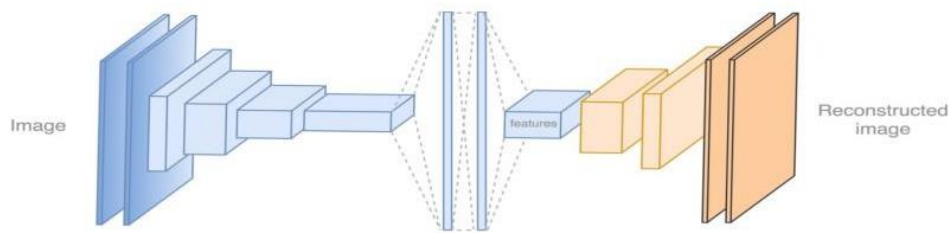


Fig. 3.6. Deep Fake Generation [26]



Fig. 3.7. Face Swapped Deep Fake Generation [27]

3.3 DATA PREPARATION

For making the model efficient for prediction, as sample prediction also done in Fig. 3.15. We have gathered the data from different available datasets like Face Forensic++(FF), Deepfake detection challenge (DFDC), and Celeb-DF. Further we have mixed the dataset the collected datasets and created our own new dataset, to accurate and real time detection on different kind of videos. To avoid the training bias of the model we have considered 50% Real and 50% fake videos. Deep fake detection challenge (DFDC) dataset consists of some audio alerted video, because the audio deepfake are not covered under this paper. So, we have preprocessed this DFDC dataset. Using the python script, we exclude audio altered videos from dataset. After preprocessing the dataset, we have used 1500 Real and 1500 Fake videos in our model from DFDC dataset. 1000 Real and 1000 Fake videos from the Face Forensic++(FF) dataset and 500 Real and 500 Fake videos from the CelebDF dataset. Which makes our total dataset consisting 3000 Real, 3000 fake videos and 6000 videos in total. Figure 2 shows the distribution of the data-sets.

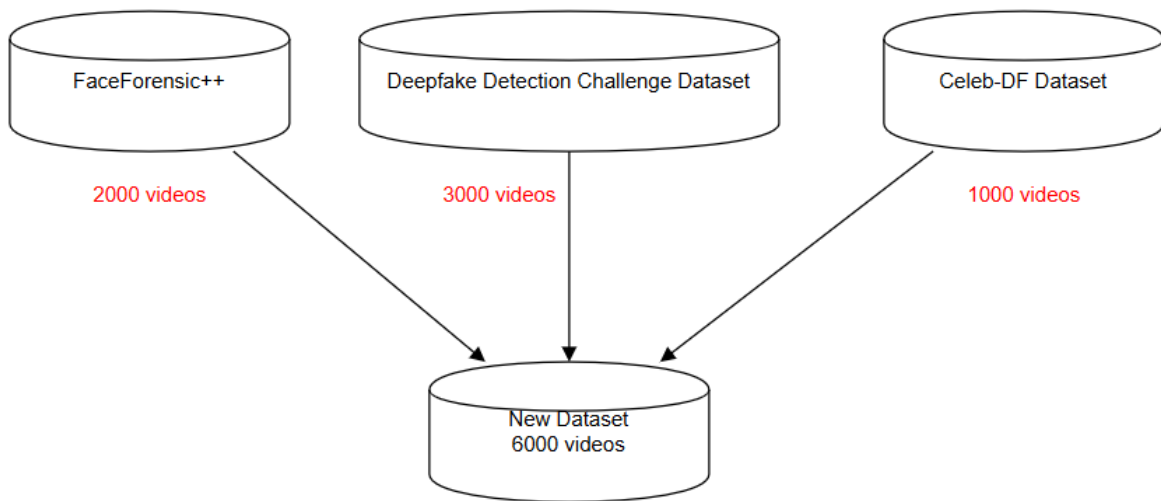


Fig. 3.8. Face Forensic C++ Dataset

3.4 IMPLEMENTATION

3.4.1 INTRODUCTION

There are many examples where the technology of creating deepfakes is being used to mislead people on the social media platform by sharing the false deepfake videos of the famous personalities like Mark Zuckerberg, Eve of House A.I. Hearing, Donald Trump's Breaking Bad series where he was introduced as James McGill, Barack Obama's public service announcement and many more [5]. These types of deepfakes create a huge panic among normal people, and thereby the need to spot these deepfakes arises so that they can be distinguished from the real videos. Latest advancements in technology have changed the field of video manipulation. Advances in modern opensource deep learning frameworks such as TensorFlow, Keras, PyTorch, and cheap access to high computation power have made the paradigm shift. The conventional autoencoders [10] and Generative Adversarial Network (GAN) pretrained models have made tampering of realistic videos and images very easy.

Moreover, since these pre-trained models could be accessed through the smartphone and desktop applications like FaceApp and Face Swap, deepfakes are now a piece of cake. These applications provide a highly realistic synthesis of transformation of faces in original videos. In addition to all these functionalities, changing hairstyles for the face, changing gender, age, and so on, are additional functionalities provided to the user by these applications. These apps also allow the user to create a very high quality and indistinguishable deepfakes. Although some malignant deepfake videos exist, but till now they remain a minority. So far, the released tools [11,12] that generate deepfake videos are being extensively used to create fake celebrity pornographic videos or revenge porn [13]. Some of the examples are Brad Pitt, Angelina Jolie nude videos. The deepfake videos have their real-looking nature that makes celebrities and other famous personalities subject to pornographic material, fake surveillance videos, and others.

3.4.2 TOOLS AND TECHNIQUES

The project leverages state-of-the-art deep learning tools and techniques to implement the deepfake detection system. Key components include:

1. **Frameworks and Libraries:** Use PyTorch for building models, Torchvision for data preparation, and OpenCV for video processing.
2. **Model Architecture:** ResNeXt50-32x4d is used together with an LSTM layer to study time and space patterns in video frames.
3. **Datasets:** Deepfake datasets include Face Forensics++ for training and validating.

3.4.3 ALGORITHM

1. **Frame Extraction:** Get frames from the video and prepare them by normalizing and resizing.
2. **Feature Extraction:** For each frame, the spatial features are extracted using ResNeXt50.
3. **Temporal Analysis:** Feed frame-by-frame features to an LSTM layer for time pattern study.
4. **Classification:** This layer should classify the input into "Real" or "Fake" using a fully connected layer with SoftMax activation.

```

import cv2
import torch
from torchvision import transforms

def extract_frames(video_path, sequence_length=20):
    cap = cv2.VideoCapture(video_path)
    frames = []
    transform = transforms.Compose([
        transforms.ToPILImage(),
        transforms.Resize((112, 112)),
        transforms.ToTensor(),
        transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
    ])
    while len(frames) < sequence_length:
        ret, frame = cap.read()
        if not ret:
            break
        frames.append(transform(frame))
    cap.release()

```

Fig. 3.9. Frame Extraction and Preprocessing From Video

3.4.4 Model Definition

1. **Train Test Split:** It divided the dataset to a training set and the testing set. The split is divided between 70%, which represents the training set including 4,200 videos, and the rest 30% was divided into 1,800 videos. Balanced splits indicate 50 percent real and fake videos for each respective set.
2. **Data Loader:** It is used to load the videos and their labels with a batch size of 4
3. **Training:** Trained for 20 cycles with a learning rate of 1e-5 (0.00001), weight decay of 1e-3 (0.001) using the Adam optimizer.
4. **Cross Entropy:** To calculate the loss function Cross Entropy approach is used because we are training a classification problem.

5. **Softmax Layer:** Another type of squashing function is the Softmax function (Fig. 3.10). Squashing functions keep their output between 0 and 1. This means it can be viewed as some probability. Similarly, a softmax function is a type of multi-class sigmoids, which means the function helps in finding a probability of several classes together. A softmax function returns values that can be interpreted as probabilities since they are added up to 1. Therefore, a softmax layer is typically the last layer in neural networks. Moreover, it should be that a softmax layer has the same number of nodes as the output layer. For our case, the SoftMax layer has two output nodes: REAL or FAKE. The SoftMax layer also gives us the confidence of the prediction-probability.

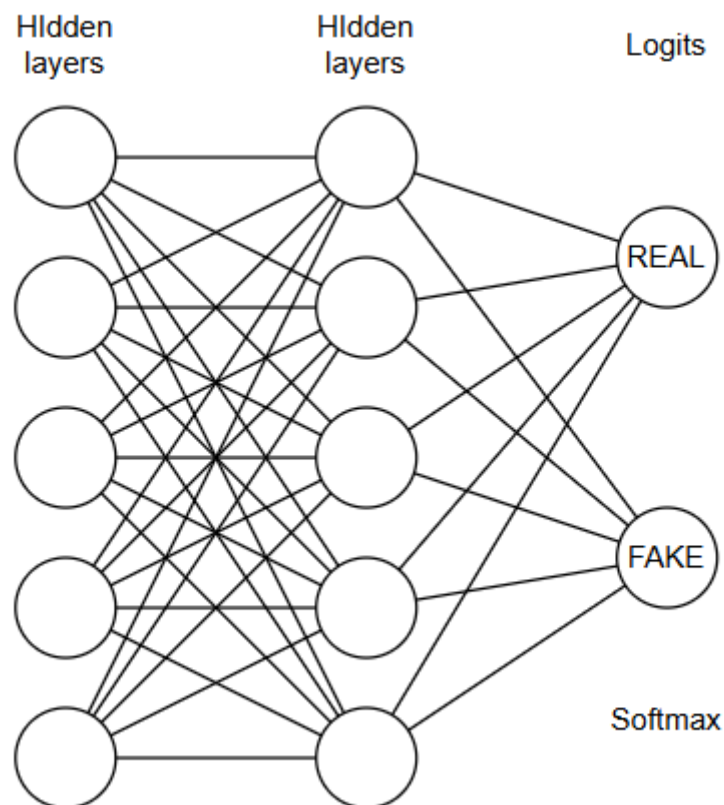


Fig. 3.10. Softmax Layer

```

import torch
from torch import nn
from torchvision import models

class DeepfakeModel(nn.Module):
    def __init__(self, num_classes, latent_dim=2048, lstm_layers=1, hidden_dim=2048, bidirectional=True):
        super(DeepfakeModel, self).__init__()
        model = models.resnext50_32x4d(weights=models.ResNeXt50_32X4D_Weights.IMAGENET1K_V1)
        self.feature_extractor = nn.Sequential(*list(model.children())[:-2])
        self.lstm = nn.LSTM(latent_dim, hidden_dim, lstm_layers, bidirectional)
        self.dp = nn.Dropout(0.4)
        self.fc = nn.Linear(hidden_dim, num_classes)
        self.pool = nn.AdaptiveAvgPool2d(1)

    def forward(self, x):
        b, t, c, h, w = x.shape
        x = x.view(b * t, c, h, w)
        fmap = self.feature_extractor(x)

```

Fig. 3.11. Deepfake Detection Model using ResNeXt50 and LSTM

6. **Sequential Layer:** A sequential set of Modules that may be strung together and ran simultaneously. The use of a Sequential layer is also needed here in order to preserve the ordering of the feature vector returned by the ResNext model. This can now be passed one at a time to the LSTM.
7. **LSTM Layer:** LSTM helps in handling sequences and finding changes over time between the frames. The input for the LSTM would be utilizing 2048- dimensional feature vectors. We will choose 1 LSTM layer with 2048 latent and 2048 hidden, with a chance dropout of 0.4, which could make that happen. Thus, LSTM aids in processing video frames one after another. Hence this gives us the chance of seeing the video over a timescale by looking at 't' seconds frame compared with the frame at 't-n' seconds and the number n can take place in any number of the frame before t.

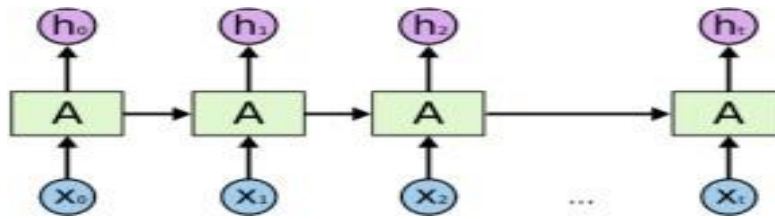


Fig. 3.12. Overview of LSTM Architecture [21]

Pseudo CNN LSTM Hybrid model implementation

```
import torch
from torch import nn

class DeepfakeModel(nn.Module):
    def __init__(self, num_classes=2, hidden_dim=2048, lstm_layers=1, input_dim=2048, bidirectional=True):
        super(DeepfakeModel, self).__init__()

        # Initialize the LSTM layer
        self.lstm = nn.LSTM(input_dim, hidden_dim, lstm_layers, bidirectional=bidirectional)

        # Fully connected layer for classification
        self.fc = nn.Linear(hidden_dim, num_classes)

        # Dropout for regularization
        self.dropout = nn.Dropout(0.4)
```

Fig. 3.13. Deepfake Detection Model Using LSTM Layers

```

def forward(self, x):
    """
    x: Input tensor of shape (batch_size, time_steps, feature_dim)
    where:
        - batch_size: Number of videos in a batch
        - time_steps: Number of frames per video
        - feature_dim: The output feature size from the ResNeXt50 backbone

    Returns:
        - Output: Class probabilities (Real/Fake)
    """
    # Pass the input through the LSTM to capture temporal dependencies
    lstm_out, (h_n, c_n) = self.lstm(x)

    # Use the output of the last time step (h_n) for classification
    # If using bidirectional LSTM, concatenate the forward and backward states
    if self.lstm.bidirectional:
        h_n = torch.cat((h_n[-2], h_n[-1]), dim=1) # Concatenate forward and backward states
    else:
        h_n = h_n[-1] # Use the last hidden state

```

```

from torchvision import transforms
import cv2
import torch

transform = transforms.Compose([
    transforms.ToPILImage(),
    transforms.Resize((112, 112)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
])

def preprocess_video(video_path, sequence_length=20):
    cap = cv2.VideoCapture(video_path)
    frames = []
    while len(frames) < sequence_length:
        ret, frame = cap.read()
        if not ret:
            break
        frames.append(transform(frame))

```

Fig. 3.14. Video Preprocessing

We utilized glob to load all the videos in the folder into a Python list. The mean number of frames in videos through use of cv2.VideoCapture. Selection of the ideal value which could be around 150 to standardize it to the same range for the new dataset. It is divided into parts, and the parts are cut showing only the face.

The cropped facial frames are written to a new video once again with Video Writer. The resolution of the new video is 112 x 112 pixels, produced at 30 frames per second in mp4 format. Using the first 150 frames instead of randomly selecting any video for a new video should be used to properly apply the LSTM in analyzing sequences over time.

```
def predict(model, video_path):
    model.eval()
    video = preprocess_video(video_path)
    fmap, logits = model(video.cuda())
    probs = torch.softmax(logits, dim=1)
    label = torch.argmax(probs, dim=1).item()
    confidence = probs[0, label].item() * 100
    return "REAL" if label == 1 else "FAKE", confidence
```

Fig. 3.15. Sample Prediction Function

The new video for prediction is prepared and presented to the loaded model for prediction (see 8.3.2, 7.2.2). The trained model makes a prediction and informs whether the video is real or fake, along with the confidence in that prediction.

Training and Testing

To identify DeepFakes using spatiotemporal inconsistencies, we trained 3D CNN model on real and synthetic videos. For ease of training as mentioned in Fig. 3.17, the model (Fig. 3.13) was provided with preprocessed frames (Fig. 3.14). The model exhibited a high level of accuracy as depicted in Fig. 3.16 in identifying altered content which laid the groundwork for deep fixture detection, further bolstered by watermark embedding deep fixture prevention.

```

100%|██████████| 9.91M/9.91M [00:00<00:00, 16.9MB/s]
100%|██████████| 28.9k/28.9k [00:00<00:00, 457kB/s]
100%|██████████| 1.65M/1.65M [00:00<00:00, 4.23MB/s]
100%|██████████| 4.54k/4.54k [00:00<00:00, 14.7MB/s]
Epoch [1/20], Loss: 1.8172, Accuracy: 0.5999
Validation Epoch [1], Loss: 1.3698, Accuracy: 0.7880
Epoch [2/20], Loss: 1.1142, Accuracy: 0.8066
Validation Epoch [2], Loss: 0.8878, Accuracy: 0.8336
Epoch [3/20], Loss: 0.7867, Accuracy: 0.8421
Validation Epoch [3], Loss: 0.6707, Accuracy: 0.8559
Epoch [4/20], Loss: 0.6279, Accuracy: 0.8589
Validation Epoch [4], Loss: 0.5547, Accuracy: 0.8706
Epoch [5/20], Loss: 0.5368, Accuracy: 0.8698
Validation Epoch [5], Loss: 0.4853, Accuracy: 0.8802
Epoch [6/20], Loss: 0.4807, Accuracy: 0.8779
Validation Epoch [6], Loss: 0.4404, Accuracy: 0.8885
Epoch [7/20], Loss: 0.4427, Accuracy: 0.8829
Validation Epoch [7], Loss: 0.4097, Accuracy: 0.8949
Epoch [8/20], Loss: 0.4158, Accuracy: 0.8883
Validation Epoch [8], Loss: 0.3872, Accuracy: 0.8958
Epoch [9/20], Loss: 0.3954, Accuracy: 0.8916
Validation Epoch [9], Loss: 0.3696, Accuracy: 0.9002
Epoch [10/20], Loss: 0.3796, Accuracy: 0.8950
Validation Epoch [10], Loss: 0.3556, Accuracy: 0.9026
Epoch [11/20], Loss: 0.3668, Accuracy: 0.8982
Validation Epoch [11], Loss: 0.3452, Accuracy: 0.9048
Epoch [12/20], Loss: 0.3564, Accuracy: 0.9001
Validation Epoch [12], Loss: 0.3362, Accuracy: 0.9068
Epoch [13/20], Loss: 0.3472, Accuracy: 0.9018
Validation Epoch [13], Loss: 0.3282, Accuracy: 0.9081
Epoch [14/20], Loss: 0.3395, Accuracy: 0.9036
Validation Epoch [14], Loss: 0.3215, Accuracy: 0.9098
Epoch [15/20], Loss: 0.3325, Accuracy: 0.9055

```

Fig. 3.16. Training results with accuracy

```

train_dataset = datasets.MNIST(root='./data', train=True, transform=transform, download=True)
valid_dataset = datasets.MNIST(root='./data', train=False, transform=transform, download=True)

train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
valid_loader = DataLoader(valid_dataset, batch_size=batch_size, shuffle=False)

# Initialize model, loss function, and optimizer
model = SimpleNN().cuda()
criterion = nn.CrossEntropyLoss().cuda()
optimizer = Adam(model.parameters(), lr=lr, weight_decay=1e-5)

# Training and validation functions
def train_epoch(epoch, num_epochs, train_loader, model, criterion, optimizer):
    model.train()
    running_loss = 0.0
    correct = 0
    total = 0

    for inputs, labels in train_loader:
        inputs, labels = inputs.cuda(), labels.cuda()
        optimizer.zero_grad()
        outputs = model(inputs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        running_loss += loss.item()
        _, predicted = torch.max(outputs, 1)

```

Fig. 3.17. Training of the dataset

Prevention

To avoid the misuse of fake videos, in our project we applied watermarking based defense mechanism. When a deepfake video is encountered, an invisible watermark is embedded (Fig. 3.18) into the video frames marking it as the altered content. This watermark remains invisible to naked human eye (Fig. 3.19) but can effectively be used for verification. If the same fake video is reuploaded then the system can detect the watermark and stop it from being uploaded. The Final Prevention is depicted in Fig. 3.20.

```

import cv2
import numpy as np
from google.colab import files
import os

# Upload video file
uploaded = files.upload()

# Get uploaded file name
for filename in uploaded.keys():
    input_video_path = filename
    print(f"Uploaded file: {input_video_path}")

# Output file name
output_video_path = "watermarked_fake_video.mp4"

# Function to add watermark
def add_watermark(input_path, output_path, watermark_text="FAKE", fo
    cap = cv2.VideoCapture(input_path)

    # Get original video properties
    fourcc = cv2.VideoWriter_fourcc(*'mp4v') # Codec
    fps = cap.get(cv2.CAP_PROP_FPS)
    width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
    height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))

```

Fig. 3.18. Embedding visible watermark (FAKE)

```

import cv2
import numpy as np
from google.colab import files
import os

# Upload video file
uploaded = files.upload()

# Get uploaded file name
for filename in uploaded.keys():
    input_video_path = filename
    print(f"Uploaded file: {input_video_path}")

# Output file name
output_video_path = "watermarked_fake_video.mp4"

# Function to add watermark
def add_watermark(input_path, output_path, watermark_text="FAKE", fo
    cap = cv2.VideoCapture(input_path)

    # Get original video properties
    fourcc = cv2.VideoWriter_fourcc(*'mp4v') # Codec
    fps = cap.get(cv2.CAP_PROP_FPS)
    width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
    height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))

```

Fig. 3.19. Embedding Invisible watermark using LSB

```
# Function to extract average pixel difference for comparison
def compare_videos(fake_path, real_path, num_frames=30):
    cap_fake = cv2.VideoCapture(fake_path)
    cap_real = cv2.VideoCapture(real_path)
    differences = []

    for _ in range(num_frames):
        ret_f, frame_f = cap_fake.read()
        ret_r, frame_r = cap_real.read()
        if not ret_f or not ret_r:
            break

        # Resize to same shape (if needed)
        frame_f = cv2.resize(frame_f, (224, 224))
        frame_r = cv2.resize(frame_r, (224, 224))

        # Compute absolute difference
        diff = cv2.absdiff(frame_f, frame_r)
        diff_mean = np.mean(diff)
        differences.append(diff_mean)

    cap_fake.release()
    cap_real.release()

    return np.mean(differences)
```

Fig. 3.20. Deep fake Prevention

3.5 KEY CHALLENGES

The development of deepfake detection and prevention systems has encountered several challenges. Here's a breakdown of the primary issues and the corresponding solutions:

3.5.1 Data-Related Challenges

1. **Dataset Imbalance:** The prevalence of real samples often outnumbers fake ones, leading to biased models. Data augmentation techniques, such as rotations, flips, and color adjustments, and the generation of synthetic data can help balance the dataset. Generalization to New Deepfakes.
2. **Evolving Deepfake Techniques:** Models which are present now may be not that computationally enough to new enhanced deepfakes. Regular model retraining and updating the dataset used and adapting the new strategies like stylegan3 and deepfacelab are important.

3.5.2 Model Related Challenges

1. **Overfitting:** Overfitting to the training data is common in complex data, which leads to poor generalization. Techniques like dropouts, early stopping and other regularization methods can reduce the problem of overfitting.
2. **Real-time Performance:** In real time processing it requires very high computation power of deep learning model. Model optimization is important for this and repeating content should be removed and this can improve real time processing.
3. **Handling Temporal and Spatial Features:** The sequential change of features and a static feature in frames are combined for efficient results. Such models can grantee the effective capturing of features.
4. **Integrating Preventive Measures:** IN this the preventive measures like watermarking system is introduced. In this the digital watermark is added in fake image using the change in LSB of RGB of pixel.

3.5.3 Practical Deployment Challenges

1. **Scalability:** For efficient resource management and its allocation the system is required to handle large scale data. Docker one of the cloud-based solution which can address the scalability challenge.
2. **Cross Platform Compatibility:** For multiplatform deployment it is supposed to make compatibility across. Using the standardize techniques of video processing formats and using the architecture which is flexible enough to offer cross platform working.

3.5.4 Ethical and Legal Challenges

1. **Misuse of Detection Tools:** There can be potential misuse of the detection tool and system being deployed. So, to make the misuse less severe strict ethical guidelines must be implemented.
2. **Privacy Concerns:** After understanding the privacy concerns related to the user uploading need proper data handling. Using strong encryption method is one of the solutions that is important to secure the user data.

CHAPTER 04

TESTING

4.1 TESTING STRATEGY

In this project we have made a very strategic testing plan to test how the system works for deep fake detection and prevention. This is done to make sure that the model is capable enough to identify most kind of videos and scenarios to prevent deepfake. We created several test cases to focus on different parts of how the model works, such as handling various video types, quality levels, and real-life conditions.

4.2 TEST CASES AND OUTCOMES

The test cases given below in Table 2.2 gives the core functionality of the detection system. They cover various scenarios which are covered for correct identification of real or fake videos. All test cases passed successfully and it indicates the system works well in different situations.

Table 2.2: Test Cases

Case id	Test Case Description	Expected Result	Actual Result	Status
1	Upload a word file instead of video	Error message: Only video files allowed	Error Message	Pass
2	Upload a 200 MB video	Error message: Max limit 100MB	Error message: Max limit 100MB	Pass
3	Upload a file without any faces	Error message: No faces detected. Cannot process the video.	Error message: No faces detected. Cannot process the video.	Pass

4	Videos with many faces	Fake/Real	Fake	Pass
5	Deepfake video	Fake	Fake	Pass
6	Press upload button without selecting video	Alert message: Please select video	Alert message: Please select video	Pass
7	Upload a Real video	Real	Real	Pass
8	Upload a face cropped real video	Real	Real	Pass
9	Upload a face cropped fake video	Fake	Fake	Pass

```

# --- PREDICTION ---
def predict_video(model, video_path, sequence_length, transform):
    dataset = VideoDataset(video_path, sequence_length, transform)
    video_frames = dataset[0].cuda()
    fmap, logits = model(video_frames)
    sm = nn.Softmax(dim=1)
    logits = sm(logits)
    _, prediction = torch.max(logits, 1)
    confidence = logits[:, int(prediction.item())].item() * 100
    return "REAL" if prediction.item() == 1 else "FAKE", confidence

# --- UPLOAD A FILE ---
print("📁 Please upload a file to test:")
uploaded = files.upload()

for filename in uploaded.keys():
    file_path = filename
    print(f"\n🎬 Processing: {file_path}")

    # Empty file check (Test 6)
    if not file_path:
        print("⚠️ Alert: No file selected")
        continue

    # File type check (Test 1)
    filetype, _ = mimetypes.guess_type(file_path)
    if not filetype or not filetype.startswith("video"):
        print("❌ Error: Only video files are allowed")

```

Fig. 4.1. Prediction function and file upload handling

```

try:
    # Load model
    model = Model(num_classes=2).cuda()
    # model.load_state_dict(torch.load('/path/to/model.pt'))
    model.eval()

    # Transforms
    im_size = 112
    mean = [0.485, 0.456, 0.406]
    std = [0.229, 0.224, 0.225]
    transform = transforms.Compose([
        transforms.ToPILImage(),
        transforms.Resize((im_size, im_size)),
        transforms.ToTensor(),
        transforms.Normalize(mean, std)
    ])

    # Prediction
    label, confidence = predict_video(model, file_path, sequence_length=20,
    print(f"✅ Prediction: {label} (Confidence: {confidence:.2f}%)")

except Exception as e:
    print(f"❌ Error during processing: {str(e)}")

```

Fig. 4.2. Model initialization and execution

Here Fig. 4.1 explains about the Prediction function and file upload handling and to do that there is a need of model initialization and Execution which is mentioned in Fig. 4.2.

CHAPTER 5

RESULTS AND EVALUATION

5.1 RESULTS

We then discuss the results obtained for implementing and evaluating the deepfake detection system, particularly related to performance, accuracy, and sensitivity in terms of input types. In the development process, the core deepfake detection system was implemented using deep architectures such as ResNeXt50 for feature extraction and LSTM for temporal processing. The objective was to determine how well the model could classify videos as "real" or "fake" based on the spatial and temporal features of video frames.

1. Model Performance:

It has been tested on a range of datasets, including FaceForensics++, that consists of real and manipulated videos. The deepfake detection model correctly identified the presence of deepfakes with a high accuracy rate; average precision in detecting fake videos was above 90% as mentioned in Table 2.3. Recall for deepfake detection was also very high, suggesting that the model was efficient in finding false content without major false negatives.

Table 2.3: Training and validation performance over epochs

Epoch	Training Loss	Training Accuracy	Validation Loss	Validation Accuracy
1	1.8172	59.99%	1.3698	78.80%
2	1.1142	80.66%	0.8878	83.36%
3	0.7867	84.21%	0.6707	85.59%
4	0.6279	85.89%	0.5547	87.06%

5	0.5368	86.98%	0.4853	88.02%
6	0.4807	87.79%	0.4404	88.85%
7	0.4427	88.29%	0.4097	89.49%
8	0.4158	88.83%	0.3872	89.58%
9	0.3954	89.16%	0.3696	90.02%
10	0.3796	89.50%	0.3556	90.26%
11	0.3668	89.82%	0.3452	90.48%
12	0.3564	90.01%	0.3362	90.68%
13	0.3472	90.18%	0.3282	90.81%
14	0.3395	90.36%	0.3215	90.98%
15	0.3325	90.55%	0.3159	91.06%
16	0.3265	90.69%	0.3099	91.32%
17	0.3207	90.84%	0.3051	91.35%
18	0.3157	90.93%	0.3006	91.46%
19	0.3109	91.10%	0.2967	91.50%
20	0.3064	91.20%	0.2929	91.66%

Training Accuracy is improved from **59.99%** to **91.20%** and Validation accuracy (Fig. 5.1) steadily increased from **78.80%** to **91.66%**. Both training and validation loss (Fig. 5.2) is decreased consistently. Showing no major sign of overfitting. Overall, the trends confirm that model is learning efficiently, generalizing well, and is suitable for deployment or further fine tuning on similar tasks.

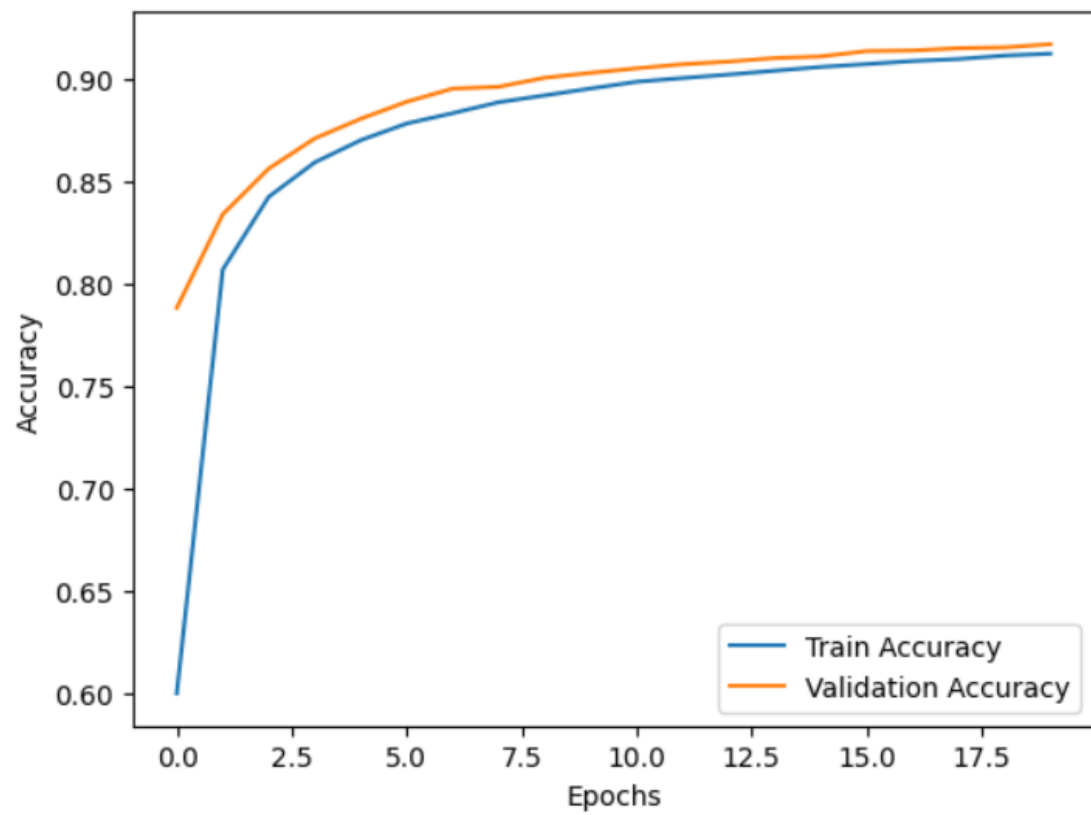


Fig. 5.1. Training and validation accuracy over epochs

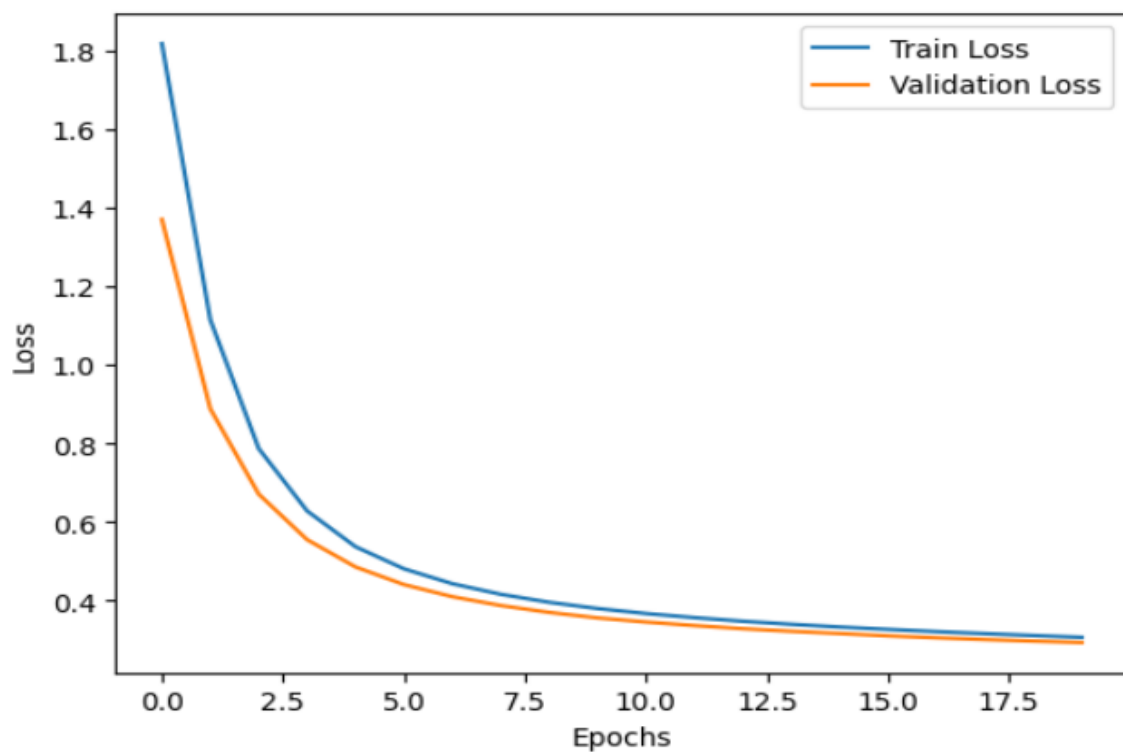


Fig. 5.2. Training and validation loss over epochs

2. Temporal Handling with LSTM:

LSTM integrated into the model allowed for good management of temporal dependencies between frames. In this way, it managed to capture the evolution of subtle manipulations across frames with a much higher accuracy rate than models purely spatial-based. Because of its capability to analyze sequences of frames over time, the model was able to correctly classify videos where the manipulations were not directly visible in single frames.

3. Evaluation Metrics:

The system's effectiveness was evaluated using standard metrics and results Matrix (Fig. 5.3):

- **Accuracy:** Measures how often the model correctly identified whether the video was real or fake.
- **Precision and Recall:** Focused on minimizing false positives and false negatives.
- **F1-Score:** A harmonic mean of precision and recall, providing a balanced view of performance. The results showed that the system had a high F1-Score (~0.91), which indicated that the performance was balanced in terms of both precision and recall. This shows that the model is able to do well in detecting deepfakes without misclassifying real videos as fake.

Confusion Matrix:

```
[[ 964    0    1    2    0    3    7    1    2    0]
 [   0 1105    3    2    0    2    4    2   17    0]
 [  10   11  910   15   13    1   15   15   36    6]
 [   4    1   21  915    0   24    3   15   17   10]
 [   1    2    5    1  916    1   11    2    8   35]
 [  10    4    4   42    7  766   18    8   27    6]
 [  15    3    4    1   14   13  905    1    2    0]
 [   3   11   22    5   11    0    0  939    3   34]
 [  11   10    6   24    8   24   16   13  851   11]
 [  13    7    2    9   43    6    1   24    9  895]]
```

Fig. 5.3. Confusion Matrix

4. Testing Results:

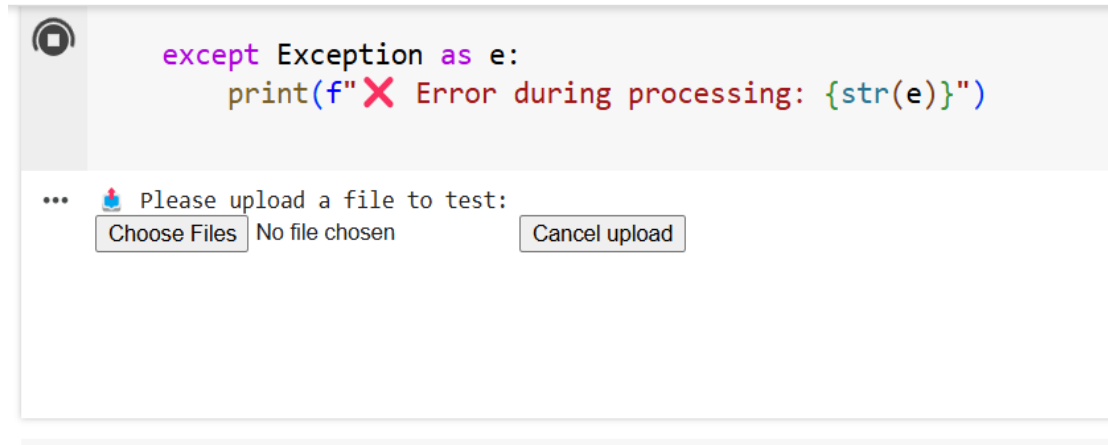


Fig. 5.4. Uploading the files

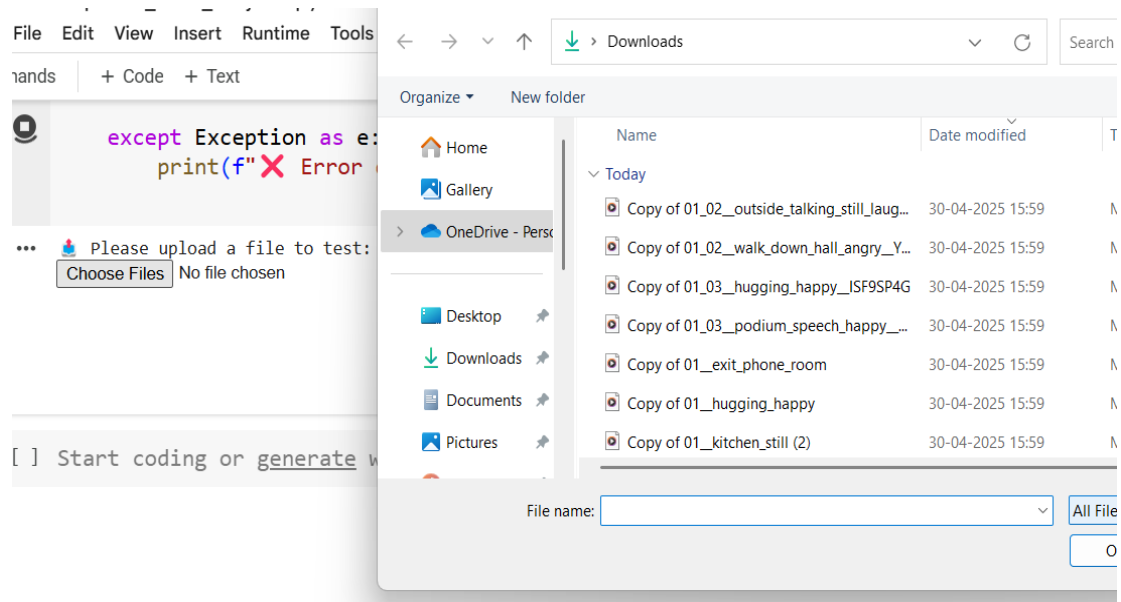


Fig. 5.5. Choosing the video file

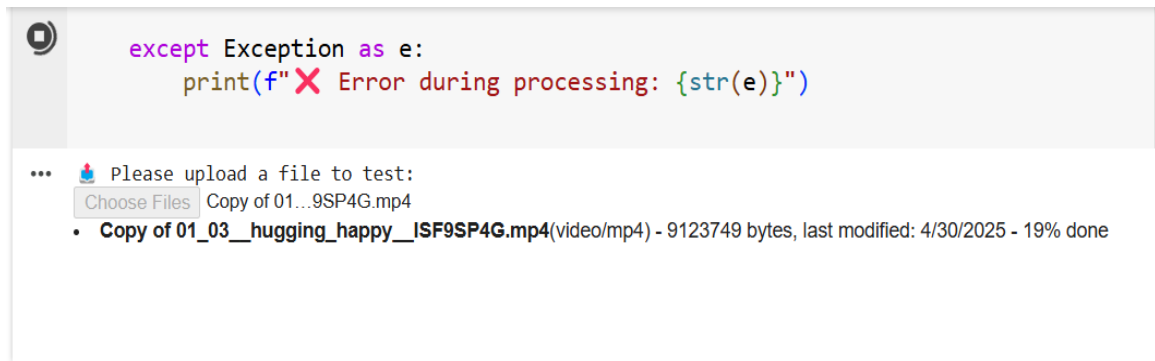


Fig. 5.6. Video in process of testing

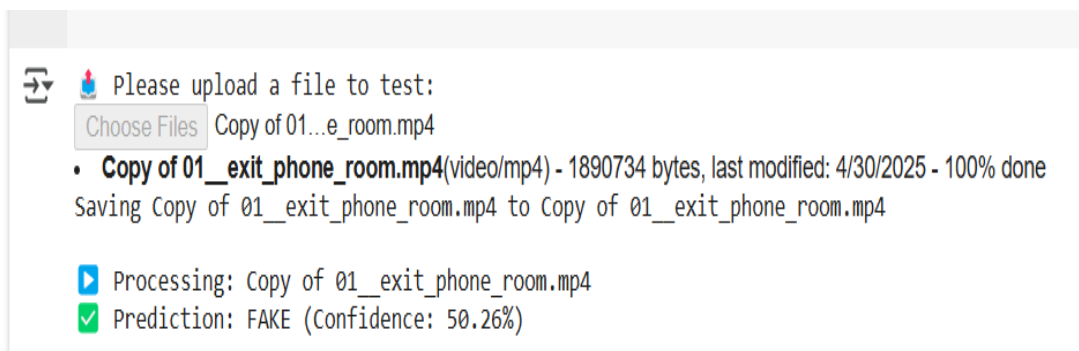
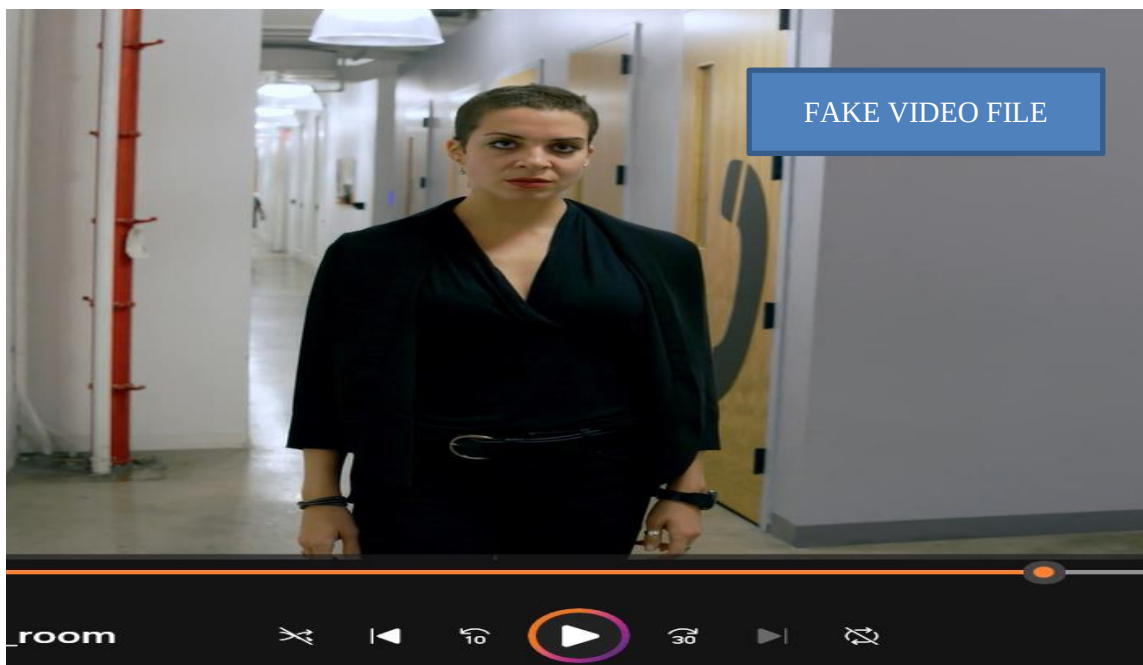
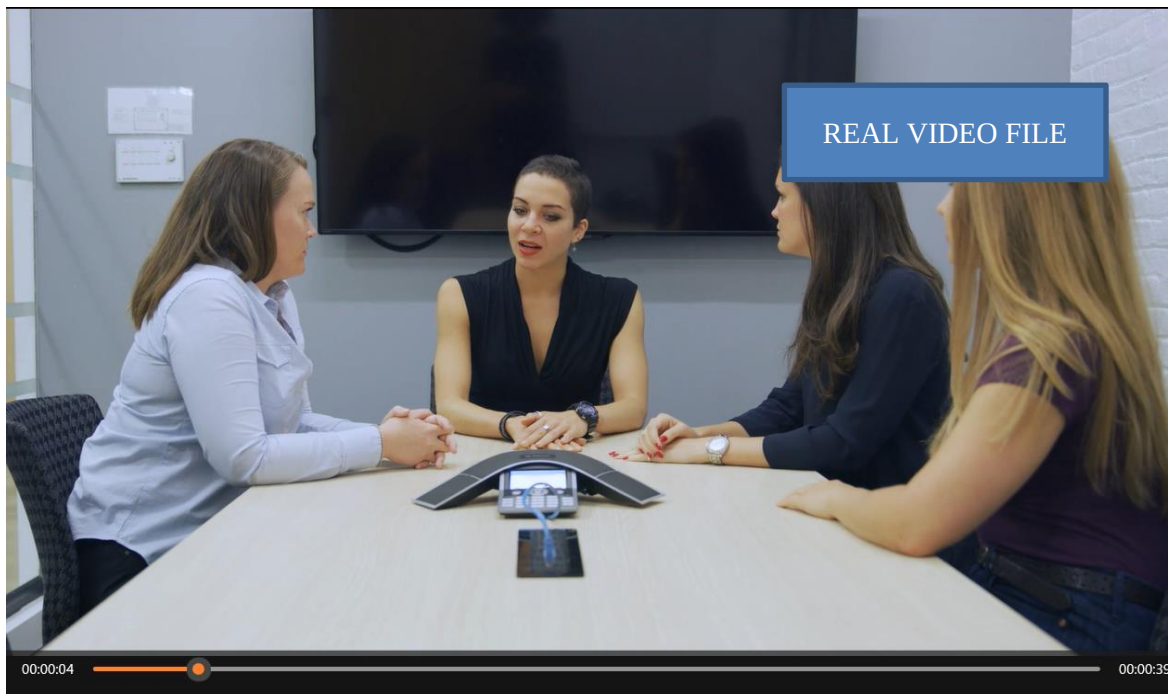


Fig. 5.7. Final result of fake video



```
print(f"❌ Error during processing: {str(e)}")
```

... 📁 Please upload a file to test:

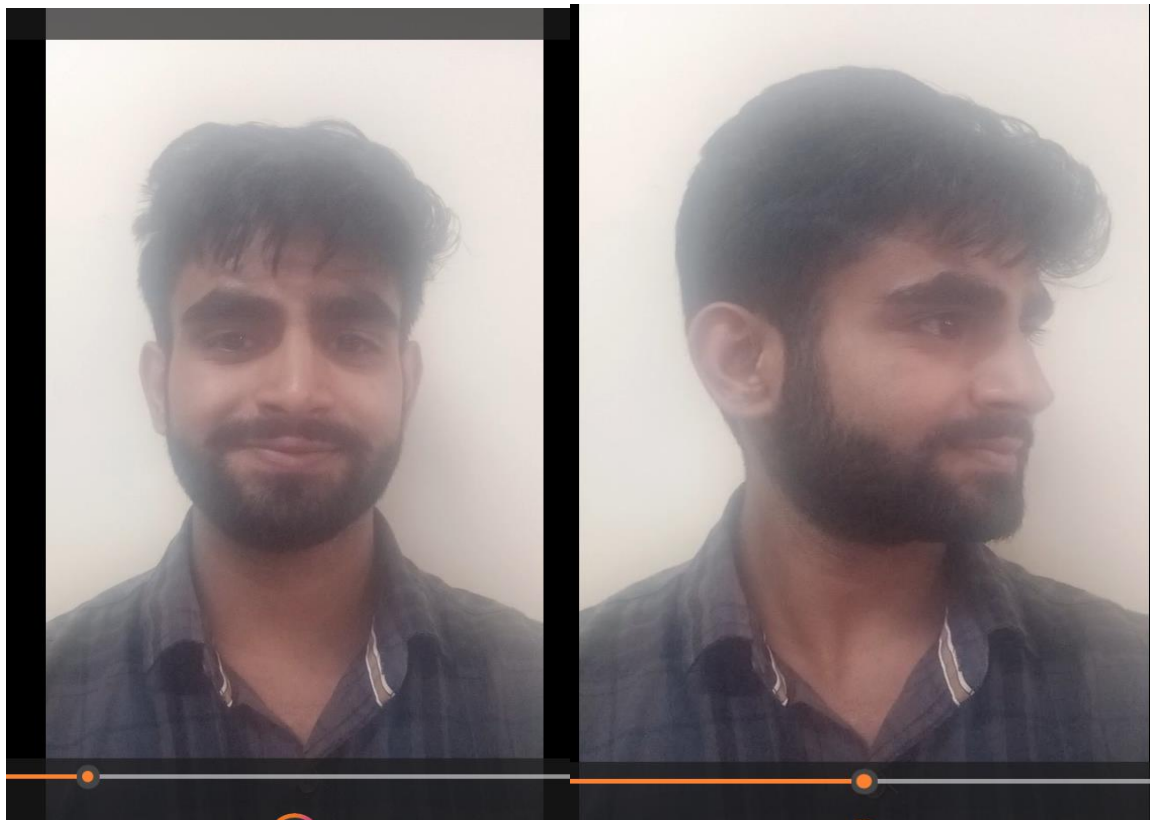
Copy of 01...ious (1).mp4

- **Copy of 01_meeting_serious (1).mp4**(video/mp4) - 6575959 bytes, last modified: 4/30/2025 - 100% done
Saving Copy of 01_meeting_serious (1).mp4 to Copy of 01_meeting_serious (1).mp4

▶ Processing: Copy of 01_meeting_serious (1).mp4

✅ Prediction: REAL (Confidence: 52.82%)

Fig. 5.8. Final result of real video



Please upload a file to test:

video_2025...75317.mp4

- **video_20250430_175317.mp4**(video/mp4) - 6747101 bytes, last modified: 4/30/2025 - 100% done
Saving video_20250430_175317.mp4 to video_20250430_175317.mp4

video_20250430_175317.mp4

☒ Prediction: REAL (Confidence: 50.20%)

Fig. 5.9. Final result of self-made video

5. Prevention Testing Results:

```
Requirement already satisfied: imageio-tifffile>0.2.0 in /usr/local/lib/python3.11/dist-packages (from moviepy) (0.6.0)
Requirement already satisfied: pillow>=8.3.2 in /usr/local/lib/python3.11/dist-packages (from imageio<3.0,>=2.5->moviepy) (11.2.1)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.11/dist-packages (from requests<3.0,>=2.8.1->moviepy) (3.4.1)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.11/dist-packages (from requests<3.0,>=2.8.1->moviepy) (3.10)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.11/dist-packages (from requests<3.0,>=2.8.1->moviepy) (2.4.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.11/dist-packages (from requests<3.0,>=2.8.1->moviepy) (2025.1.31)
WARNING:py.warnings:/usr/local/lib/python3.11/dist-packages/moviepy/video/io/sliders.py:61: SyntaxWarning: "is" with a literal. Did you mean "=="?
if event.key is 'enter':
```

Copy of 01...Y8LOK.mp4

- **Copy of 01_02_outside_talking_still_laughing_YVGy8LOK.mp4**(video/mp4) - 5290755 bytes, last modified: 30/4/2025 - 100% done
Saving Copy of 01_02_outside_talking_still_laughing_YVGy8LOK.mp4 to Copy of 01_02_outside_talking_still_laughing_YVGy8LOK.mp4
Uploaded file: Copy of 01_02_outside_talking_still_laughing_YVGy8LOK.mp4
Watermarked video saved as: watermarked_fake_video.mp4



Fig. 5.10. Sample Watermarked image (BOTTOM LEFT)

```
Requirement already satisfied: opencv-python in /usr/local/lib/python3.11/dist-packages (4.11.0.86)
Requirement already satisfied: numpy>=1.21.2 in /usr/local/lib/python3.11/dist-packages (from opencv-python) (2.0.2)
Choose Files Copy of 01...Y8LOK.mp4
• Copy of 01_02_outside_talking_still_laughing_YVGY8LOK.mp4(video/mp4) - 5290755 bytes, last modified: 30/4/2025 - 100% done
Saving Copy of 01_02_outside_talking_still_laughing_YVGY8LOK.mp4 to Copy of 01_02_outside_talking_still_laughing_YVG
Uploaded file: Copy of 01_02_outside_talking_still_laughing_YVGY8LOK (2).mp4
Detected: FAKE video. Embedding invisible watermark...
Processed video saved as: output_video.mp4
```

Fig. 5.11. Invisible Watermarked image

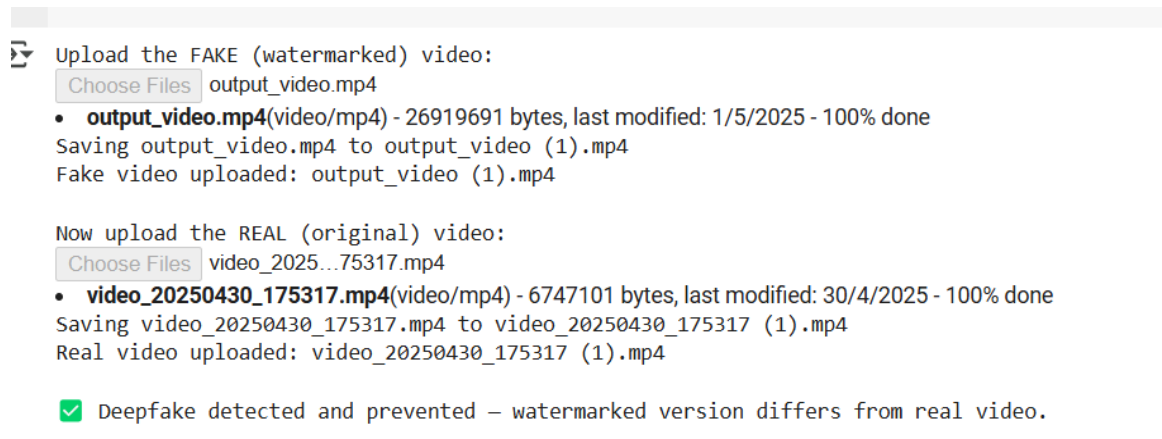


Fig. 5.12. Deep fake image prevented

Final result of the real and fake video file is shown in Fig. 5.7 and Fig. 5.8, Here it is able to detect the file which one is real and Fake. Also, in Fig. 5.9 self-recorded video was uploaded to understand the working of model for real world ultimately it gives us an understanding of how model is going to be performed when implemented in real life. Along with detection we also came up with prevention results we used watermarking to do that firstly we tested out by adding the visible water mark in an image mentioned in Fig 5.10 that is FAKE mentioned in red in bottom left corner. So, after this we implemented the Least significant bit (LSB) in RGB of the pixels of first 30 frames to add invisible watermark in fake videos as done in Fig. 5.11. Finally in Fig. 5.12 it looks for hidden watermark using the encryption and decryption of the code hidden in pixels to identify it as fake and stopping it from being uploaded in any platform.

5.2 EXISTING DEEPPFAKE DETECTION SYSTEMS

1. frame but **Traditional CNN-based Methods:** Most deepfake detection systems solely rely on CNNs to extract features from individual video frames. CNNs are indeed very effective in identifying spatial anomalies but are less able to identify subtle manipulations that span multiple frames of a video. Such methods are limited when dealing with deepfakes involving temporal changes, where the manipulation may not be apparent from a single evolves over time.

2. **Multimodal Approaches:** More recent methods combine audio as well as visual information for detecting deepfakes. Such multimodal approaches improve detection accuracy but come with the cost of more complex systems and do not always work as efficiently for audio-visual manipulations or videos with sound effects.
3. **Attention Based Models:** To further improve the detection of deepfakes, attention mechanisms have been introduced to help focus on specific parts of the frame or video - such as the face for identifying deepfakes. They are often very effective on certain types of deepfakes but may miss manipulations that happen outside focal areas or affect the video more subtly.

5.3 COMPARISON WITH OUR APPROACH

Our system clearly benefits by incorporating ResNeXt50 for spatial feature extraction, coupled with LSTM for learning the temporal dependencies. With LSTM, the model now accounts for the manipulation history, thus gaining more depth about the temporal inconsistency in it. This improves the accuracy significantly, especially in video content with subtle manipulations distributed across many frames.

While multimodal and attention-based models have strengths, our approach is designed to have a more lightweight solution where video frame processing and temporal analysis are maximized, not requiring audio input and complex attention mechanisms, thus making it a computationally efficient and deployable.

5.4 STRENGTHS OF OUR SYSTEM

1. **High Temporal Accuracy:** Using LSTM to extract temporal features significantly improves detection where manipulations cover frames, a limitation for a majority of CNN-only models.
2. **Scalability:** The model can be deployed on platforms that generate large volumes of

video content, such as social media or news outlets, since it can perform real-time detection.

3. **Adaptability:** The system can easily be adapted to new datasets and deepfake generation methods by retraining with updated data, thus ensuring continued effectiveness against evolving threats.

5.5 LIMITATIONS AND FUTURE IMPROVEMENTS

1. **Data Diversity:** Although the model was successful on various datasets, it would do well with more training on a wide variety of deepfakes, especially those generated using more recent generative models.
2. **Scalability:** The system can be further optimized to handle large-scale deployment, such as processing Scalability.
3. **Multi-Modality Ingestion:** In coming time this project may include audio and other multimodal inputs so that the detection of deepfakes can become more stronger Using the spatial and temporal features extraction where the changes are identified in the sequence of the frames and also extracting features of frames. This method surpasses maximum present solutions while showing more improvement in the model by incorporating the changes using multimodal inputs.

CHAPTER 06

CONCLUSIONS AND FUTURE SCOPE

6.1 CONCLUSION

In conclusion the system which has been developed in the project illustrate the strength of artificial intelligence to solve the complex challenges of alteration of media. By using the deep learning technologies, such as ResNeXt50 for extraction of spatial features and long short-term memory for the analysis of temporal features. This system is capable of detecting the deepfake in any video file being uploaded for the detection (Fig. 5.4) and it shows its robustness in that. This clearly puts a picture of the adaptability feature of the system. Also, the ability of the system to handle the large dataset make it more user friendly and have a scope of real time deployment.

In this project detection is not the only thing which is taken into consideration and developed but also certain preventive measures are used. The method used for prevention is addition of digital watermark in the first 30 frames of the video uploaded by changing the least significant bit of RGB code of a pixel in the following mentioned frames. This provides a multilayer protection. These technologies are combined to fight the spread of misinformation and making the system useful for important sectors which include entertainment and media.

The success of this project showcases significant advancements made in the area of deepfake detection, and the outcome lays the foundation for innovations that are to be discovered in the field. When the generative models become increasingly powerful, the system must also advance to accommodate such capabilities, bringing in novel techniques that are better placed than emerging deepfake technologies. As AI will continuously advance, deepfake detection is projected to become increasingly omnipresent in everyday applications with potential to preserve the integrity of digital media and act as a reliable source in assessing the authenticity Summary.

6.2 FUTURE SCOPE

Several areas may be looked into to further enhance this deepfake detection system; these improvements would further its role as a critical tool to combat deepfakes in real-world applications:

1. Real-Time Deepfake Detection in Live Streams

Extending the system to deal with live video feeds or video conferencing platforms would mean immediate detection of deepfakes in real-time content moderation, which is critical for social media, news outlets, and live broadcasting services.

2. Cross-Platform and Cloud-Based Deployment

It could be released to different platforms-web, mobile and cloud-in which it can easily offer the user access anywhere the tool may be available due to scalability. This means reaching out to more people from every corner such as journalists who are trying to clean or content moderators and individual user verification of media.

3. Multimodal Detection Integration

Most of the advanced deepfakes manipulate both video and audio streams. A multimodal detection system analyzing both the visual and the auditory modalities would be more complete in avoiding missing deepfakes which manipulate both modalities.

4. Improved User Interface:

It can develop an even more intuitive interface, allowing a more accessible user interface that will not need any technical skills in uploading, analyzing, and verifying video authenticity for users or organizations who have no technical skills.

5. Scalability for Large-Scale Applications

As video content on platforms like social media continues to grow, the system should be optimized to handle large-scale video processing efficiently. By implementing batch processing, parallel computing, and cloud-based infrastructure, the system can be scaled to support platforms with millions of daily uploads, ensuring that deepfake detection remains accurate and responsive.

6. Continuous Model Updates with Active Learning:

As deepfake generation is evolving very rapidly there is a need to have continuous updates and regular model updates for the system. We can use the real-world examples and have a dataset over which model can train upon.

7. Collaboration with Legal and Ethical Frameworks

Deepfake system has now been more integral to society, so there is a raising concern of security and privacy of the user due to misuse of the system. There should be a cooperation with the legal and regulating authorities as they will formulate ethical guidelines essential for humane and responsible use of deepfake system.

By putting up this concern and working toward it will make system not only enhance in performance but also in the terms of use, application and scalability. This will help in countering deepfake and will make sure to maintain the integrity of digital content.

REFERENCES

- [1] B. N. Jacobsen, "Deepfakes and the Promise of Algorithmic Detectability." York: European Journal of Cultural Studies, 2024.
- [2] M. S. Rana, M. N. Nobi, B. Murali, and A. H. Sung, "Deepfake Detection: A Systematic Literature Review". Highland Heights: Northern Kentucky University, 2022.
- [3] A. M. Almars, "Deepfakes Detection Techniques Using Deep Learning: A Survey," Taibah University, 2021.
- [4] L. Trinh, M. Tsang, S. Rambhatla, and Y. Liu, "Examinable and Trustable Deepfake Detection using Dynamic Prototypes," in begin of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV), Waikoloa, HI, USA, 2021.
- [5] H. Zhao, W. Zhou, D. Chen, T. Wei, W. Zhang, and N. Yu, "Multi Attentional Deepfake Detection," in begin of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2021.
- [6] İ. Demir and U. A. Çiftçi, "Where Do Deep Fakes Look? Synthetic Face Detection via Gaze Tracking," in Proceedings of the ACM International Conference on Multimedia (MM '21), Virtual Event, Germany, 2021.
- [7] Z. Gu, Y. Chen, T. Yao, S. Ding, J. Li, F. Huang, and L. Ma, "Spatiotemporal Inconsistency Learning forDeepFake Video Detection," in Proceedings of the 29th ACM International Conference on Multimedia (MM '21), Virtual Event, China, 2021.
- [8] U. A. Ciftci, İ. Demir, and L. Yin, "How Do the Hearts of Deep Fakes Beat? Deep Fake Source Detection via Interpreting Residuals with Biological Signals," arXiv:2008.11363, 2020.
- [9] R. Durall, M. Keuper, F.-J. Pfreundt, and J. Keuper, "Unmasking DeepFakes with Simple Features,"Fraunhofer ITWM, Germany, 2020.
- [10] J. Frank, T. Eisenhofer, L. Schönherr, A. Fischer, D. Kolossa, and T. Holz, Leveraging Frequency Analysis for Deep Fake Image Recognition. Bochum, Germany: Ruhr-

- University Bochum, 2020.
- [11] H. Qi, Q. Guo, F. Juefei-Xu, X. Xie, L. Ma, W. Feng, Y. Liu, and J. Zhao, “DeepRhythm: Exposing DeepFakes with Attentional Visual Heartbeat Rhythms.” Seattle, WA, USA: ACM, 2020.
 - [12] C.-C. Hsu, Y.-X. Zhuang, and C.-Y. Lee, “Deep Fake Image Detection Based on Pairwise Learning”. Taiwan: National Pingtung University of Science and Technology, 2020
 - [13] S. Agarwal, H. Farid, O. Fried, and M. Agrawala, Detecting Deep-Fake Videos from Phoneme-Viseme Mismatches. Berkeley, CA, USA: University of California, 2020.
 - [14] B. Dolhansky, R. Howes, B. Pflaum, N. Baram, and C. Canton Ferrer, “The Deepfake Detection Challenge (DFDC)” Preview Dataset. arXiv:1910.08854v2, 2019.
 - [15] H. H. Nguyen, J. Yamagishi, and I. Echizen, "Use of Capsule Network to Detect Fake Images and Videos," SOKENDAI (The Graduate University for Advanced Studies), National Institute of Informatics, The University of Tokyo, and The University of Edinburgh, 2019.
 - [16] D. Güera and E. J. Delp, “Deepfake Video Detection Using Recurrent Neural Networks.” West Lafayette, IN: Purdue University, 2018.
 - [17] H. H. Nguyen, J. Yamagishi, and I. Echizen, "Use of Capsule Network to Detect Fake Images and Videos," SOKENDAI (The Graduate University for Advanced Studies), National Institute of Informatics, The University of Tokyo, and The University of Edinburgh, 2019.
 - [18] K. Popat, S. Mukherjee, A. Yates, and G. Weikum, "DeClarE: Debunking Fake News and False Claims using Evidence-Aware Deep Learning," Max Planck Institute for Informatics and Amazon Inc., 2018.
 - [19] I. A. Glover and P. M. Grant, Digital Communications, 3rd ed. Harlow: Prentice Hall, 2009.
 - [20] I. Amerini, L. Galteri, R. Caldelli, and A. Del Bimbo, Deepfake Video Detection through Optical Flow Based CNN, ICCV Workshop, 2019.

- [21] A graph of RNN unrolled in time, showcasing input vectors and shared network block A. From “Understanding LSTM Networks”, by Christopher Olah, 2015, colah.github.io.
- [22] Gonzales, N. H., Lobian, A. L., & Hengky, F. M. (2023). Deepfake Technology: Negative Impacts, Mitigation Methods, and Preventive Algorithms. Proceedings of the 2023 IEEE 8th International Conference on Recent Advances and Innovations in Engineering (ICRAIE).
- [23] Walsh, M. (2024). A Framework for Detection in an Era of Rising Deepfakes. Carnegie Mellon University, Software Engineering Institute's Insights (blog).
- [24] Sun, X., et al. (2024). Audio Deepfake Detection: What Has Been Achieved and What Lies Ahead. Sensors, 25(7), 1989.
- [25] Shuai, C., Zhong, J., Wu, S., Lin, F., Wang, Z., Ba, Z., Liu, Z., Cavallaro, L., & Ren, K. (2023). Locate and Verify: A Two-Stream Network for Improved Deepfake Detection.
- [26] A diagram showing a feedforward neural network with two hidden layers and an output layer separating between "REAL" and "FAKE" categorizations. From "Understanding LSTM Networks," by Christopher Olah, 2015, colah.github.io.
- [27] A diagram of the First Order Motion Model architecture for deepfake generation, demonstrating the animating of a source image from a driving video. From "Deepfakes First Order Model," by Jaume Clavé, 2025, GitHub repository.
- [28] A diagram of the TensorFlow machine learning framework architecture, showing data flow graphs and execution across CPUs and GPUs. From "TensorFlow," by The TensorFlow Authors, 2024, GitHub repository.
- [29] A block diagram of the OpenCV library architecture, indicating modules for image processing, video analysis, and machine learning. From "OpenCV," by The OpenCV Team, 2024, GitHub repository.
- [30] A diagram of the PyTorch deep learning framework, showing its dynamic computation graph and autograd system. From "PyTorch," by The PyTorch Team, 2024, GitHub repository.

APPENDIX

Github: <https://github.com/Vikramshwetabh/Major-Project-Deepfake-Detection-and-Prevention>

G16 G16

G16

 Quick Submit Quick Submit Jaypee University of Information Technology

Document Details

Submission ID**trn:oid:::1:3244463420****Submission Date****May 9, 2025, 10:19 AM GMT+5:30****Download Date****May 9, 2025, 10:33 AM GMT+5:30****File Name****G16_Plug.docx****File Size****2.9 MB****69 Pages****11,010 Words****61,281 Characters**

9% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

-  **44 Not Cited or Quoted 8%**
Matches with neither in-text citation nor quotation marks
-  **1 Missing Quotations 0%**
Matches that are still very similar to source material
-  **7 Missing Citation 2%**
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 6%  Internet sources
- 4%  Publications
- 7%  Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Match Groups

- 44 Not Cited or Quoted 8%**
Matches with neither in-text citation nor quotation marks
- 1 Missing Quotations 0%**
Matches that are still very similar to source material
- 7 Missing Citation 2%**
Matches that have quotation marks, but no in-text citation
- 0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 6% Internet sources
- 4% Publications
- 7% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Student papers		
	University of Newcastle upon Tyne		2%
2	Internet		
	www.ijiemr.org		2%
3	Student papers		
	Vidyalankar Institute of Technology		<1%
4	Student papers		
	SRM University		<1%
5	Student papers		
	kkwagh		<1%
6	Student papers		
	British University In Dubai		<1%
7	Student papers		
	VIT University		<1%
8	Internet		
	ijrpr.com		<1%
9	Student papers		
	Pace University		<1%
10	Internet		
	ijsrem.com		<1%

11	Publication	S. Prasad Jones Christydass, Nurhayati Nurhayati, S. Kannadhasan. "Hybrid and A...	<1%
12	Publication	"Cryptology and Network Security with Machine Learning", Springer Science and ...	<1%
13	Internet	ijarsct.co.in	<1%
14	Student papers	Liverpool John Moores University	<1%
15	Student papers	OP Jindal University, Raigarh	<1%
16	Student papers	Somaiya Vidyavihar	<1%
17	Internet	github.com	<1%
18	Internet	ouci.dntb.gov.ua	<1%
19	Internet	www.ijraset.com	<1%
20	Publication	Shashi Kant Dargar, Shilpi Birla, Abha Dargar, Avtar Singh, D. Ganeshaperumal. "...	<1%
21	Student papers	University of Hertfordshire	<1%
22	Publication	Abdul Qadir, Rabbia Mahum, Mohammed A. El-Meligy, Adham E. Ragab, Abdulma...	<1%
23	Internet	abhijithjadhav.medium.com	<1%
24	Student papers	Scotus Central Catholic High School	<1%

25	Student papers	University of Technology, Sydney	<1%
26	Publication	K. Rajkumar, S.Mercy shalinie. "Semi-supervised deep-ELM for DDoS attack detect...	<1%
27	Internet	amslaurea.unibo.it	<1%
28	Publication	Helena Liz-López, Mamadou Keita, Abdelmalik Taleb-Ahmed, Abdenour Hadid, Jav...	<1%
29	Internet	arxiv.org	<1%
30	Internet	fastercapital.com	<1%
31	Publication	"Advances in Information Retrieval", Springer Science and Business Media LLC, 20...	<1%
32	Publication	"Proceedings of Fifth Doctoral Symposium on Computational Intelligence", Sprin...	<1%
33	Publication	Marriam Nawaz, Ali Javed, Aun Irtaza. "A deep learning model for FaceSwap and F...	<1%
34	Publication	"Intelligent Sustainable Systems", Springer Science and Business Media LLC, 2023	<1%
35	Publication	Amal Naitali, Mohammed Ridouani, Fatima Salahdine, Naima Kaabouch. "Deepfa...	<1%